

PRESERVING VERIFIED OUTPUT MODES IN RLVR: MODEBENCH AND REPLAYMAXRL

Anonymous authors

Paper under double-blind review

ABSTRACT

A model can learn to answer a problem more reliably while losing alternative correct solutions. Reinforcement learning with verifiable rewards (RLVR) provides little feedback about this loss: a repeated success and a different success receive the same reward. We introduce MODEBENCH, five executable domains that identify distinct verified outputs for the same prompt, and ReplayMaxRL, which combines learning from fresh samples with memory of verified successes. The same execution keys serve both measurement and replay: each retained key receives equal weight, even when it becomes rare in fresh rollouts. Across three model scales, adding replay to Dr.GRPO improves held-out correctness and sampled verified support. Replay also complements MaxRL at both scales with complete comparisons; the available 3B results and harder-level comparisons show where this evidence remains incomplete. Hosted-model evaluation also finds accurate answers concentrated on few verified modes. An idealized analysis explains why binary rewards can amplify common modes and how replay can counter that pressure. The results separate success from preserving verified alternatives.

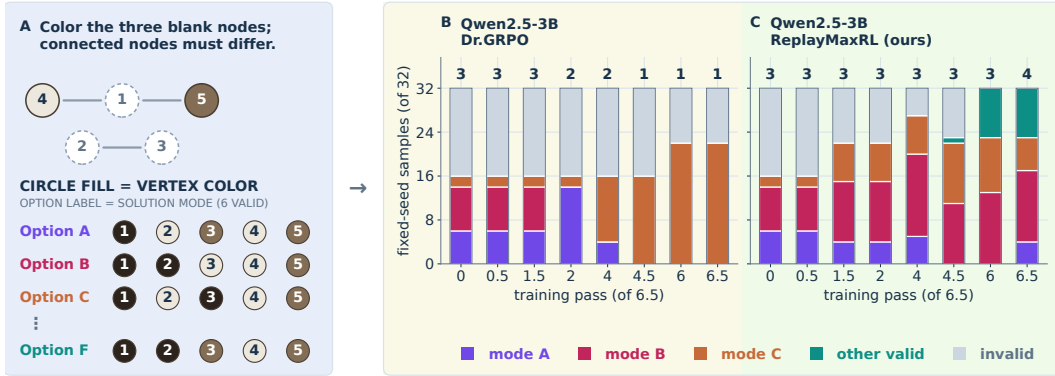


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; ReplayMaxRL produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Figure 5 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A mathematical agent often works by proposing a computation or proof route, checking it, and trying again after feedback (Brown et al., 2024; Snell et al., 2024). A different route may offer a way forward when the first fails or the problem’s constraints change. A verifier can assess these alternatives only if the model still proposes them. This motivates a question about training: as a model becomes more successful, what happens to the range of successful behaviors it can produce?

Reinforcement learning with verifiable rewards (RLVR) uses executable correctness feedback to improve a model (Guo et al., 2025). That feedback does not distinguish a familiar correct so-

lution from another correct solution to the same problem. The model can therefore become more accurate while its verified outputs concentrate on fewer alternatives. Figure 1 shows the distinction on one prompt. Across our benchmark, verifier-only training reduces the number of sampled correct modes beyond the first at all three model scales, although correctness and raw mode counts need not decline in every comparison (Appendix F.1).

The missing measurement is the identity of a successful output. Different strings may encode the same computation, while different computations may reach the same answer. Neither token entropy nor answer accuracy alone tells us which verified alternatives remain available. MODEBENCH addresses this by having each validator return a canonical execution key alongside correctness. These keys define task-specific output modes, allowing evaluation to measure whether sampling succeeds and how many distinct verified outputs it finds.

The same identity also gives training a useful memory. ReplayMaxRL stores one verified exemplar per admitted key and assigns equal replay weight to each retained key. MaxRL learns from fresh samples; replay continues to train on stored successes when they become rare in those samples. Ordinary success replay can reuse correct responses without canonical keys. Our use of keys makes distinct verified outcomes measurable and prevents repeated discoveries of a common outcome from dominating the memory’s weights.

We make three connected contributions: an executable benchmark for measuring verified alternatives, a method that uses those identities for balanced replay, and matched experiments that separate the fresh learning objective from memory. The evidence supports a practical claim: replay can improve both held-out correctness and sampled verified breadth, including beyond a stronger fresh objective. Whether preserving these alternatives improves later search, revision, or adaptation remains a question for further work.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

For a prompt x with specification s_x , a validator maps an output y to failure $\perp$ or a canonical key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs count as the same mode exactly when their keys agree. Training sees only keys generated by the current policy; neither the full valid-key set $\mathcal{C}_x$ nor its size is supplied to the learner or replay bank.

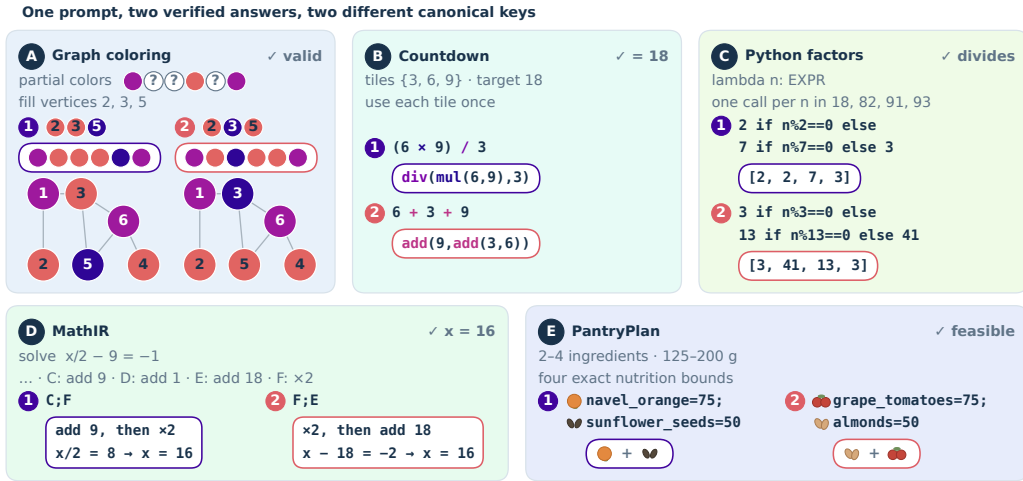


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

Figure 2 shows the five domains. Countdown and MathIR capture different routes to an answer, Python captures different tested behaviors, and Graph and PantryPlan capture different valid configurations.

This distinction matters when interpreting breadth: an additional valid configuration and an additional computation route are both verified modes, but they are not the same kind of alternative. The keys measure executed outputs, not latent reasoning strategies or the usefulness of a solution in a later task. Appendix B gives the exact key and alias rules, with verbatim prompts in Appendix C.

2.1 SUCCESS AND SAMPLED BREADTH

For K independent outputs, let D_K count their distinct valid keys. Our two primary metrics ask complementary questions:

$$\text{pass@}K = \Pr[D_K > 0], \quad \text{distinct@}K = \mathbb{E}[D_K].$$

The first measures the chance of finding any correct output; the second measures how many different correct outputs the samples contain. We use $K = 8$ throughout the training comparisons. The metrics have different units: probability and expected verified-key count. Their numerical magnitudes should not be compared directly, and a larger mode count can partly reflect more successful samples rather than a more balanced correct-output distribution.

To make that coupling visible, we also report *extra modes*, $\text{distinct@}8 - \text{pass@}8$: the expected number of verified keys beyond the first correct output in a sampled set. This decomposition helps interpret a breadth gain but does not hold correctness fixed. None of these finite-sample measurements identifies the total number of positive-probability modes or proves that an unobserved mode has become impossible. Appendix A gives the full definitions and reference policies.

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

The benchmark has two disjoint levels for the training experiments. Level 2 adds structural difficulty while retaining each domain’s validator, key, training/evaluation sizes, and valid-mode-count histograms on those splits. For example, Graph hides more vertices and Countdown requires more operands. Frozen-model evaluation checks that the harder level remains solvable. This lets us change task difficulty without changing what counts as a distinct success; the construction and admission checks appear in Appendix B.3.

3 REPLAYMAXRL: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

ReplayMaxRL uses the two outputs of the validator for different purposes: correctness rewards train on fresh samples, and canonical keys organize memory. Both paths use policy-generated responses. The method needs neither gold solutions nor a precomputed catalogue of valid modes.

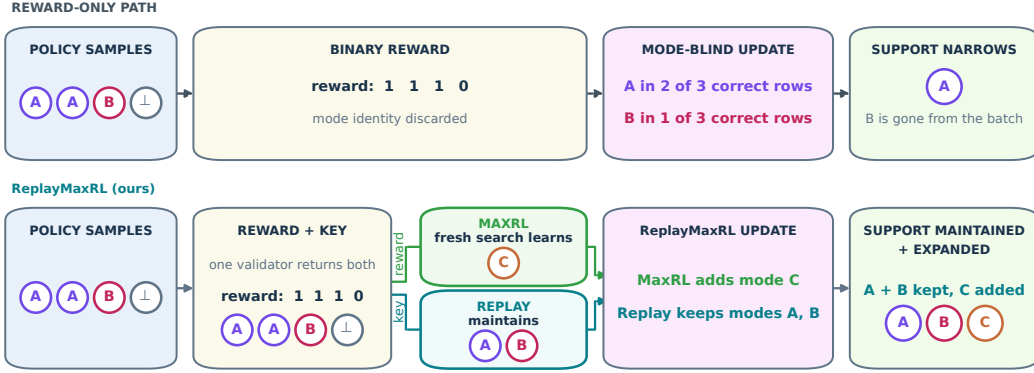


Figure 3: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores exemplars by canonical key and revisits retained keys with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

3.1 LEARNING FROM FRESH ROLLOUTS

MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026). With binary rewards, its centered group estimator weights a correct response more strongly when successes in the group are scarce. This provides a useful test of replay: memory should be evaluated alongside an objective designed for learning from sampled successes, as well as alongside Dr.GRPO (Liu et al., 2025).

Both fresh objectives still depend on what the policy currently samples. An absent mode contributes no direct score term, and an all-correct or all-incorrect group has zero centered task advantage. MaxRL changes the fresh learning signal without creating a persistent record of which correct behaviors have occurred. Appendix D.1 specifies the exact advantages and their finite-group interpretation.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Each training prompt owns an initially empty, bounded bank $\mathcal{B}_x$. When a new key is admitted, the bank retains one verified exemplar e_c . Repeated discoveries of that key do not add further entries. A deterministic schedule revisits nonempty prompt banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (1)$$

Each retained key receives the same share of replay, independent of how often it appeared in fresh rollouts. A mode discovered once therefore continues to receive a signal alongside a mode discovered repeatedly. Token normalization sets the loss scale for each exemplar; it does not make exemplar likelihood equal to the total probability of generating its canonical key.

We add this loss to the fresh MaxRL update. Replacing MaxRL with Dr.GRPO yields ReplayDr.GRPO. Replay exemplars stay outside the fresh reward group, so the fresh objective and replay can be varied separately. The controls execute the same bank and scoring code with zero replay gradient; their learned banks and workloads can differ as policies diverge. Banks contain only responses to training prompts. Held-out evaluation therefore tests what training changes in the policy, rather than retrieving answers from the bank. Appendix D gives admission, scheduling, masking, and loss scaling.

3.3 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

Common correct modes appear in more on-policy updates than rare correct modes, although both receive the same binary reward. In an isolated softmax model with independently trained logits, the

mean updates of GRPO, Dr.GRPO, and binary MaxRL amplify an initial imbalance and generically concentrate on one correct mode. A fixed replay bank changes this flow: persistent likelihood training supplies a qualitative probability floor for its stored modes (Appendix I).

Theory studies idealized mean flows. The result explains a pressure that replay can counter; it does not certify the implemented neural optimizer. A finite bank protects only admitted modes in the ideal analysis, and its probability floor can be too small to yield observable samples. Our experiments therefore measure held-out support directly and separately inspect fixed-exemplar score trajectories. They do not infer universal preservation from the theorem.

4 EXPERIMENTAL DESIGN

We compare Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B on all five Level-1 domains. Each run trains for eight passes with five registered seeds per comparison. Evaluation averages four eight-sample draws on 128 held-out prompts, with matched requests across methods. Complete paired blocks receive nominal 95% intervals; incomplete intersections retain their exact seed counts and are descriptive. Missing or ambiguous endpoints are not imputed. The shared recipe and source-integrity rules are in Appendices B.4 and J.1.

The comparisons answer three questions. First, does adding canonical replay to Dr.GRPO improve correctness and verified support across scales? Supporting controls test reward normalization, on-policy redistribution, ordinary success memory, and semantic exploration. UCPO and sparse RLEP-Dr are compared at 0.5B and 1B only; fixed Semantic-MaxEnt is defined in Appendix D.5. Second, does replay still help when the fresh objective is MaxRL? The central factorial crosses Dr.GRPO/MaxRL with replay off/on at all three scales, yielding Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL. Third, does the same pattern hold on the harder Level 2? This factorial uses Qwen2.5-0.5B, with four complete domain comparisons and partial PantryPlan results. Appendix Table 4 gives the exact coverage.

5 RESULTS

5.1 EXPERIMENT 1: REPLAY IMPROVES SUCCESS AND SAMPLED SUPPORT

Replay improves success and sampled support across scales. Figure 4A shows positive block-mean changes in both primary endpoints for every model and domain. Raw `distinct@8` increases in all 74 admissible seed pairs. This consistency matters: the result spans valid configurations, computation routes, and tested program behaviors, rather than depending on a single task or favorable seed. The excluded Falcon Countdown run and its four-seed comparison remain explicit in the figure and Appendix J.1.

Countdown provides a concrete interpretation: replay more than triples the number of sampled verified computation routes at Qwen2.5-0.5B while also raising correctness. Python shows why the two metrics must be read together: its correctness improves substantially, but the gain in breadth beyond the first correct output remains uncertain. More successful samples alone do not demonstrate a more balanced distribution over correct outputs. The appendix reports the exact means and uncertainty for these examples (Appendix E.2).

The controls also distinguish the value of balancing keys from simply reusing successes. At Qwen2.5-0.5B, ReplayDr.GRPO has the largest mean correctness gain among the direct alternatives in every domain (Figure 4B). Replacing equal key weights with weights proportional to fresh discovery frequency reduces extra modes on average, while the aggregate correctness contrast remains inconclusive. The advantage is concentrated in Graph, Countdown, and PantryPlan; it is not a uniform benefit across tasks. This ablation supports key balancing as a contributor to breadth (Appendix E.6).

5.2 EXPERIMENT 2: REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Replay adds value beyond MaxRL at both scales with complete MaxRL pairs. The two replay comparisons in Figure 5 separate memory from the choice of fresh objective. At 0.5B and 1B, replay improves the domain-averaged correctness and raw mode count under both Dr.GRPO

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

Δ pass@8

Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28
	Graph	Countdown	Python	MathIR	Pantry	Average

Δ distinct@8

Qwen 0.5B	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56
	Graph	Countdown	Python	MathIR	Pantry	Average

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

Δ pass@8

Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03
	Graph	Countdown	Python	MathIR	Pantry	Average

Δ distinct@8

Before training	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04
	Graph	Countdown	Python	MathIR	Pantry	Average

Figure 4: **Canonical replay improves correctness and verified breadth.** (A) ReplayDr.GRPO minus Dr.GRPO at all three scales. (B) Qwen2.5-0.5B alternatives minus Dr.GRPO. Columns measure probability points and expected verified-key counts, respectively. The green Average weights domains equally within seed. A dagger marks four paired seeds in Falcon Countdown and its Average; other cells use five. Only complete blocks receive nominal 95% intervals. Purple denotes ReplayDr.GRPO; bold entries mark column maxima in Panel B.

and MaxRL. Thus the replay benefit persists when fresh learning already uses MaxRL’s sampling-aware objective. The comparison identifies the effect of adding the replay update; it does not establish which individual stored modes account for the held-out gains.

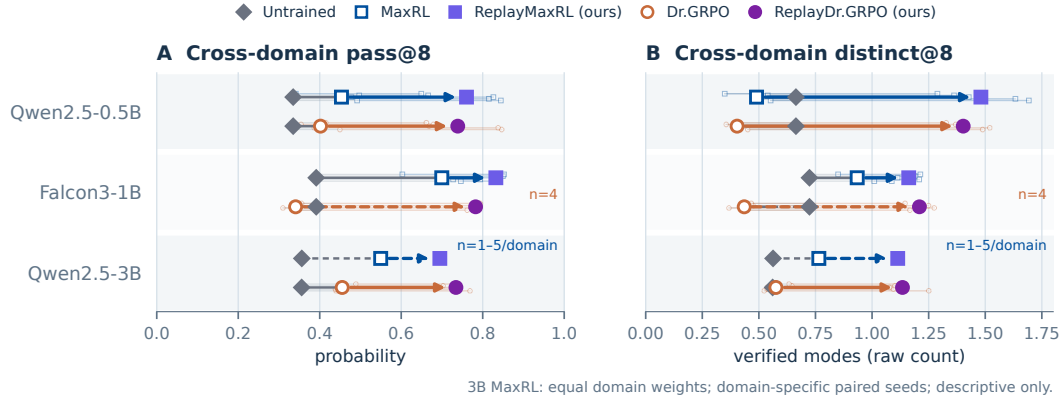


Figure 5: **Replay complements both fresh objectives at multiple scales.** Each row shows Dr.GRPO/ReplayDr.GRPO and MaxRL/ReplayMaxRL on both endpoints. Diamonds are initial models; open/filled markers are training without/with replay. Complete tracks average domains within seed; faint paths show seeds. Falcon Dr.GRPO uses four common seeds. The dashed 3B MaxRL track averages domain-specific paired means equally ($n = 5, 3, 5, 3, 1$ for Graph/Countdown/Python/MathIR/Pantry), with no common-seed trajectory or interval. Appendix Figure 8 reports each domain’s correctness and verified-mode results at every scale.

The effect is not identical across domains or scales. ReplayMaxRL improves both endpoints in every 0.5B domain. At 1B, mean correctness improves in every domain, but Graph’s mean raw-support effect is negative and inconclusive. The 3B Dr.GRPO comparison is complete. For 3B MaxRL, Graph and Python are complete while the other domains have smaller paired cohorts. The descriptive 3B average also favors ReplayMaxRL on both endpoints. We retain all three model rows in the main

figure, while distinguishing this partial summary from the complete scale comparisons. Appendix E.2 gives the aggregate estimates; Appendix E.4 shows both metrics throughout training.

5.3 EXPERIMENT 3: REPLAY ON HARDER MATCHED TASKS

Replay improves both average endpoints at both difficulty levels. Figure 6 compares the four domains with complete factorials at Qwen2.5-0.5B. Their aggregate moves toward greater correctness and sampled support with replay under either fresh objective, at both levels. The levels preserve the mode definition and the distribution of valid-mode counts while changing task structure, so the comparison tests more than a different key definition. PantryPlan remains outside this aggregate because its Level-2 factorial is incomplete.

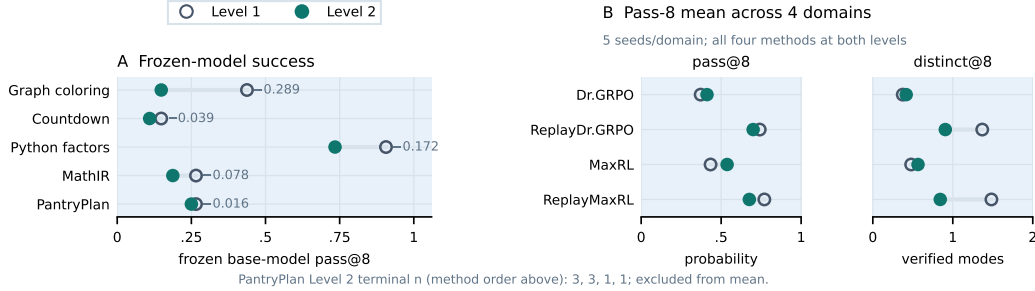


Figure 6: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2. (A) Frozen-model success in all five domains. (B) Terminal means over Graph, Countdown, Python, and MathIR, weighted equally within each of five paired seeds. Both levels and all four methods share this completed cohort. Partial PantryPlan arm counts are reported separately (Appendix E.3).

Graph supplies the clearest domain-level evidence: both replay variants improve correctness and raw support substantially on the harder problem. The other completed domains’ correctness and raw-support intervals include zero. We therefore interpret the aggregate as evidence that replay can remain useful under this controlled increase in difficulty, not as a universal gain on harder tasks. Appendix E.3 reports the Graph estimates, and Appendix E.3 gives all fresh-objective and replay-interaction contrasts.

5.4 HOSTED MODELS: CORRECTNESS DOES NOT DETERMINE BREADTH

The distinction measured by MODEBENCH also appears beyond the training comparisons. We evaluate seven hosted deployments on all five domains and three levels, without additional training. Figure 7 places per-response correctness beside the number of verified modes found in eight samples. Graph and MathIR show why both matter: highly accurate outputs often revisit only one or a few verified modes. PantryPlan yields broader sampled outputs, with substantial variation across deployments and levels.

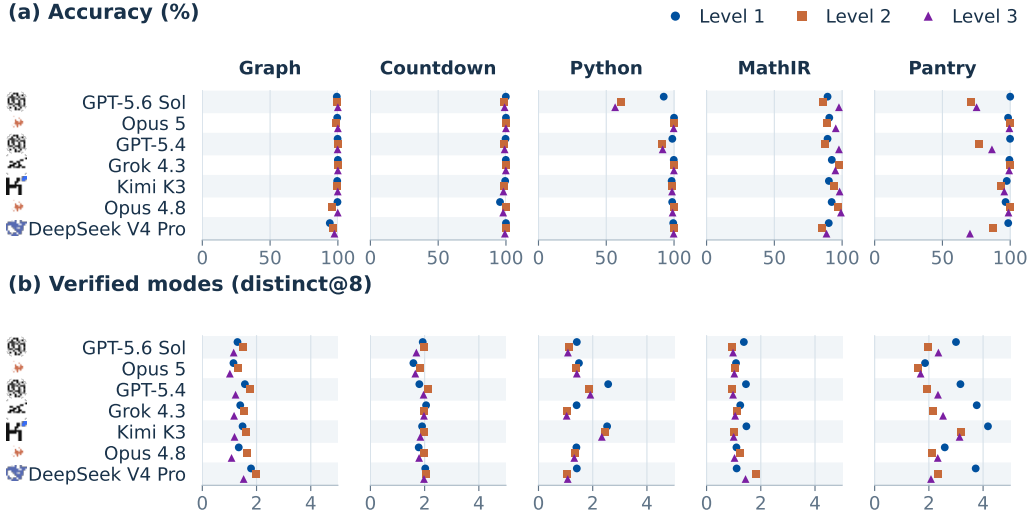


Figure 7: **Accurate hosted outputs can still cover few verified modes.** Columns show domains; markers distinguish Levels 1–3. Each point uses all eight responses to each of 128 prompts. Top: per-response accuracy. Bottom: distinct@8 . A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. Appendix H gives original-protocol results, prompt comparisons and uncertainty.

These inference-only comparisons do not identify a training cause or a model-size effect. Benchmark levels need not become harder for every deployment.

6 RELATED WORK

RLHF and reasoning RL can reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). MODEBENCH measures verified output identities; semantic clustering instead infers equivalence from language (Farquhar et al., 2024). Sampling-based selection motivates preserving proposal alternatives (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024); our experiments do not measure their downstream utility.

Outcome bonuses, set prediction, distribution matching and uniform-correct objectives also encourage broader support (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026). MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); we ask whether memory adds value. Replay can mitigate forgetting (Lin, 1992; Rolnick et al., 2019), and RLEP reuses successful reasoning trajectories (Zhang et al., 2025). Our memory balances execution keys independently of discovery frequency, connecting replay to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates correctness from the availability of alternative verified outputs. Replay-MaxRL uses their execution identities as memory: fresh updates improve the policy on new samples, while replay keeps stored successes involved in training. Matched comparisons show that replay can improve correctness and sampled breadth, including beyond MaxRL.

Our claims concern sampled, task-defined outputs. Replay cannot retain a mode before discovering and storing it, and improved held-out breadth does not show that every stored exemplar survives. A fixed-bank audit finds improving central scores alongside a declining tail (Appendix F.3); exact mode probabilities remain unverified. The idealized theory does not certify neural training, and the experiments do not show that breadth causes the accuracy gains. Whether retaining alternatives helps agents recover or adapt remains an open question.

REFERENCES

- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025. URL <https://arxiv.org/abs/2505.22617>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.
- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.

- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6820–6829. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017. URL <https://arxiv.org/abs/1701.06548>.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou. RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.

APPENDIX ORGANIZATION

The appendix gives metric and objective details (Appendix A), then follows the experiment’s dependencies: benchmark and evaluation protocol (Appendix B), replay algorithm and comparator definition (Appendix D), current results (Appendix E), mechanism checks (Appendix F), core proofs (Appendix I), and source integrity and reproducibility (Appendix J). The separate hosted-model evaluation provides inference-only context. Historical experiments and extended theory remain in the linked research archive.

A METRIC AND OBJECTIVE DETAILS

Correctness and solution support are distinct properties of a policy. For a prompt x with reference specification s_x , the validator maps a response to either failure or a canonical key:

$$V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}.$$

Here $\mathcal{C}_x$ is the set of valid solution keys and $\perp$ denotes failure. The learner observes a key only for a response it generates; neither the full set $\mathcal{C}_x$ nor its size is revealed.

For $Y \sim \pi_\theta(\cdot \mid x)$, the policy induces both a total probability of verified success and a distribution over how that probability is allocated:

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c \mid x) = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp].$$

The conditional distribution is defined only when $P_\theta^+(x) > 0$; when $P_\theta^+(x) = 0$, all three sampling metrics below are zero. The first quantity measures correctness. The second describes the distribution of verified behaviors conditional on being correct. We call concentration of p_θ^+ onto fewer keys *mode collapse*, particularly when P_θ^+ remains stable or increases.

A.1 MEASURING SUCCESS AND SUPPORT

For K independent samples $Y_{1:K}$, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

MODEBENCH reports three averages with different units. $\text{distinct@}K = \mathbb{E}[D_K]$ is an expected count in $[0, K]$ of unique verified keys; $\text{pass@}K = \Pr[D_K > 0]$ and $\text{mean@}K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$. For every draw, $F_K \leq \mathbf{1}[D_K > 0] \leq D_K/K$; hence $\text{mean@}K \leq \text{pass@}K \leq \text{distinct@}K/K$. The inequality is valid, but equal numerical changes are not commensurate: .02 in $\text{pass@}8$ is two probability points, whereas .02 in $\text{distinct@}8$ is .02 expected verified keys.

For a fixed prompt, write $\mu_c(x) = \Pr[V(Y, s_x) = c]$. Independence gives

$$\text{pass@}K(x) = 1 - (1 - P_\theta^+(x))^K, \quad \text{distinct@}K(x) = \sum_{c \in \mathcal{C}_x} [1 - (1 - \mu_c(x))^K].$$

These identities apply before averaging over prompts; applying the nonlinear formula to an average success probability generally gives a different result.

Raw $\text{distinct@}K$ measures sampled coverage, not the total number of positive-probability modes. For an uninformed reference, suppose a finite space has N actions and n_c actions map to valid key c . Uniform independent action sampling then gives $\sum_c [1 - (1 - n_c/N)^K]$ expected verified keys. The simpler expression $m_x[1 - (1 - 1/N)^K]$ applies when the m_x valid actions have distinct keys, as in our reported finite-action references. Open-ended program and derivation spaces have no canonical nominal “uniform” generator.

The registered terminal co-primary endpoints are $\text{pass@}K$, the probability of finding any correct output, and raw $\text{distinct@}K$, the number of verified keys found. We report both separately for each model–domain–level block. The derived contrast $\text{distinct@}K - \text{pass@}K = \mathbb{E}[D_K - \mathbf{1}[D_K > 0]]$ is an interpretive decomposition: it estimates the number of verified modes beyond the first. It is computable without knowing the full valid set $\mathcal{C}_x$ and remains coupled to the number of successful draws. We use it as an interpretive decomposition rather than a fully success-matched diversity measure or a third primary endpoint. Because every headline evaluation fixes $K = 8$, reporting $\text{distinct@}8/8$ would add no information beyond a linear rescaling; we retain the raw count and label its units explicitly.

A.2 WHY BALANCE WITHIN THE REPLAY BANK

Uniform sampling over keys prevents discovery frequency from determining replay frequency. To explain its relationship to the evaluation metric, we first analyze an equal-weight categorical surrogate,

without length normalization. A stored response’s probability need not equal its key’s total probability. For $C = |\mathcal{B}_x|$ banked keys, write $p(c)$ for their probabilities and q_B for their total mass. Then

$$-\frac{1}{C} \sum_{c \in \mathcal{B}_x} \log p(c) = -\log q_B + \log C + \text{KL}(U_C \parallel p(c \mid c \in \mathcal{B}_x)). \quad (2)$$

The objective rewards greater banked verified mass and a more balanced conditional distribution, aligning with `pass@K` and `distinct@K`. The combined update need not improve both at every step. Indeed banked-key coverage contributes $\sum_c [1 - (1 - p(c))^K]$ to expected `distinct@K`; at fixed q_B and C , concavity makes it largest under uniform allocation, while $1 - (1 - q_B)^K$, the probability of any banked key, depends only on total mass. This deliberate objective–metric alignment motivates treating held-out `pass@K` as co-primary. Uniformity also maximizes the smallest conditional banked-key mass, making equal retention explicit. Banks contain only training-prompt responses; evaluation responses never enter training.

B BENCHMARK AND EVALUATION PROTOCOL

MODEBENCH defines correctness and identity through the same executable validator: invalid outputs receive reward zero and no key; valid outputs receive reward one and a canonical execution key. Neither the complete valid-key catalogue nor its size is supplied to training, replay, scheduling, or stopping. Table 1 specifies the five domains and which surface forms merge. The public dataset preserves each domain’s split roles and construction identity. This paper’s training comparisons use Levels 1 and 2; the separate Level-3 release does not imply completed Level-3 training results.

Table 1: **MODEBENCH**. Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt, exact over the released multi-answer split. Every domain trains on 384 prompts and evaluates on a fixed 128-prompt split; the exact prompt each policy receives is reproduced verbatim in Appendix C for a direct, byte-level audit of the evaluation interface.

Domain	Generated answer / check	Executed key	Aliases that merge	valid modes
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19

B.1 SPLIT IDENTITIES AND EXECUTABLE VALIDATION

Every domain uses 384 training and 128 disjoint evaluation prompts. Python materialization independently executes at least two functions with different valid keys for every row; MathIR executes every certified route through the same interpreter used for model outputs.

Table 2: Deterministic materialization identities. Hashes bind the released rows to their executable checks; the full hashes and construction records accompany the data.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96
Gold catalogue supplied to policy	no	no
Training-bank initialization	empty	empty

PantryPlan. The deterministic v2 release has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8 to 45; the 128 evaluation prompts contain 2,328 feasible supports, averaging 18.1875. Training and evaluation row hashes are a4883c5e...a757f and 5d81ba8d...5923. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

B.2 PYTHON EXECUTION CONTRACT

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean `and/or`, unary plus/minus/`not`, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and other identifiers are rejected. Task specifications contain two to eight distinct integers in $[4, 1000]$ and at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm; the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield $\perp$ and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For every frozen input n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor`: $d_1, \dots, d_m$; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

B.3 MATCHED LEVEL-2 CONSTRUCTION

Level 2 changes problem structure while preserving the Level-1 verifier, canonicalizer, training/evaluation sizes, and exact training/evaluation histograms of valid-mode counts. Its 384/128/128 training/development/evaluation identities are disjoint from every Level-1 and Level-2 split. Development histograms also match, except that Pantry scales its 64-row Level-1 histogram by two for 128 Level-2 rows. Thus the evaluated comparison changes difficulty without changing the amount of valid support.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

Before treatment training, frozen Qwen2.5-0.5B evaluation admitted a Level-2 domain only when `pass@8` lay in $[.10, .90]$ and was strictly below its Level-1 value under identical decoding. The released matched-level identity manifest records support histograms, row hashes, construction seeds, disjointness checks, and these admission receipts.

B.4 TRAINING AND EVALUATION SETTINGS

Table 3: **Matched run contract.** Settings and systems changes are shared by methods within each model–domain–seed comparison, including the zero-weight replay control.

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .

Base model	Pinned revision
Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfd
Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

Responses are independent within each evaluation draw. For each training seed, metrics average over all 128 prompts and four draws; raw responses and draw-level records are retained. The co-primary terminal outcomes are `pass@8` and `raw distinct@8`; `mean@8` is diagnostic. Evaluation never supplies training data, bank entries, checkpoint selection, or a stopping rule. Complete paired five-seed blocks receive nominal pointwise 95% Student- t intervals; partial primary intersections report exact counts and descriptive means without intervals. Intervals condition on the frozen prompts and evaluation requests. Domain averages are explicitly labeled, and no missing terminal result is replaced by a shallower checkpoint. Source admission and curve gaps follow Appendix J.1; comparison-specific intersections and the replay-weighting bootstrap are specified with their results.

C EXACT DOMAIN PROMPTS

Each domain uses one system message and one user message, with no few-shot examples. The same template renders training and evaluation for every method. These are the first rows of the frozen evaluation splits, including the exported chat-template wrapper. The checked generator `ops/check_paper_domain_prompts.py` reproduces them from the datasets. A two-space continuation indent marks display wrapping: joining it to the previous line with one space recovers the original string. The PantryPlan listing retains its real newlines.

C.1 GRAPH COLORING

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/graph_coloring_modebench_v2`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3), (3,5),
(3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no edge has the
same color on both endpoints. A partial coloring is 2??1?2, where ? means uncolored.
Keep every shown digit fixed. There are exactly 3 question marks. Return exactly one
```

```

final answer and no explanation: exactly 3 digits, each one 1, 2, or 3, for the missing
positions from left to right, inside \boxed{<|im_end|>
<|im_start|>assistant

```

C.2 COUNTDOWN

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/exact_countdown_easy3_probe.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use each
given number exactly once. You may use +, -, *, /, and parentheses. Return exactly one
final answer and no explanation: the expression inside \boxed{<|im_end|>
<|im_start|>assistant

```

C.3 PYTHON FACTORS

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/python_factor_modebench_v1.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external Python tool
will call it once for each n in [18, 82, 91, 93]. For every call, return an integer d
satisfying 1 < d < n and n % d == 0. Different valid return vectors are different
solution modes. You may use integer literals, n, +, -, *, //, %, comparisons, Boolean
operators, and conditional expressions; calls, imports, attributes, containers, and
other names are forbidden. Return exactly the one-line lambda inside \boxed{<|im_end|>
explanation.<|im_end|>
<|im_start|>assistant

```

C.4 MATHIR ACTION MENU

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/mathir_action_menu_v1.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed by
exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c. Action
menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E means
sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations, repeated
states, and paths that do not finish with x isolated are rejected. Return only the IDs
separated by semicolons inside the answer box; for example formatting only: \boxed{A;C}.
Do not return x's numeric value.<|im_end|>
<|im_start|>assistant

```

C.5 PANTRYPLAN

Template qwen_pantry_support_mask; 384 training and 128 evaluation
rows in var/data/pantry_plan_modebench_v2.

```

<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the corresponding
Pantry row in printed order. A trusted environment will choose exact quantities on
precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:

```

```

- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal, protein=21.5g,
  fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal, protein=0.74g,
  fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal, protein=0.941g,
  fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
  includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant

```

D REPLAY ALGORITHM

For each prompt x , $\mathcal{B}_x$ stores at most M verified policy exemplars, one per canonical key; the serialized cursor τ selects banks for replay. The same update supports either fresh objective.

D.1 FRESH TASK ADVANTAGES

For a group of G fresh responses with verified binary rewards r_i and $m = \sum_i r_i$, Dr.GRPO uses $A_i = r_i - m/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} Gr_i/m - 1, & m > 0, \\ 0, & m = 0. \end{cases}$$

The centered MaxRL estimator (Tajwar et al., 2026) gives failures advantage -1 in mixed groups and weights each success more strongly when successes are scarce. All-correct and all-failure groups have zero task advantage. Lemma I.2 gives its harmonic pass@ k mean objective. These advantages apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

D.2 STATE AND OPTIMIZER UPDATE

Algorithm 1 One ReplayMaxRL optimizer update

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \tau)$

Ensure: Atomically updated $(\theta', \mathcal{B}', \tau')$

- 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward r_i
- 4: Set $c_i \leftarrow \tilde{c}_i$ iff $r_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 5: **end for**
- 6: Compute fresh task advantages A_i^o from $\{r_j\}_{j=1}^G$; for ReplayMaxRL use the formula in Appendix D.1
- 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
- 8: $(z, \tau') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \tau)$ by a fixed round-robin cursor
- 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
- 10: For each retained exemplar e_c , compute $s_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} s_c$
- 12: **else**
- 13: $L_{\text{replay}} \leftarrow 0$
- 14: **end if**
- 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$
- 17: Checkpoint $(\theta', \mathcal{B}', \tau')$ as one atomic state

D.3 ADMISSION, CAPACITY, AND DETERMINISTIC SCHEDULING

A bank is keyed by the SHA-256 digest of the exact prompt-token sequence. Only loss-active rows with positive reward and a nonempty validator key qualify. The full group is validated against the pre-group bank before any new key is committed. For each new key, the lexicographically smallest normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is therefore invariant to rollout-row order. The bank keeps one exemplar per key, up to $M = 16$, with no eviction or replacement. Later keys may enter diagnostics but not replay support when capacity is full. Prompt-hash collisions or changed tokens under an existing hash are hard errors.

At each update, the scheduler sorts nonempty prompt-bank hashes and selects one by a fixed round-robin cursor; singleton banks are eligible. The cursor is independent of reward, evaluation, desired support, or entropy, and is serialized with the bank. Stored prompt and response tokens are materialized unchanged into a right-padded teacher-forcing batch. Prompt and padding positions are masked. Each exemplar contributes its mean response-token log probability, averaged uniformly across keys in the selected bank before applying λ_{replay} and the exact $(G - 1)/G^2$ rollout scaling in Algorithm 1. Neither exhaustive support nor evaluation outputs enter this loss.

D.4 MATCHED CONTROLS AND RECOVERABLE STATE

Controls share validation, bank updates, scheduling, materialization, and the score pass, with the replay score-gradient vector replaced by exact zeros. Treatment activates only that derivative. Policies can subsequently differ in samples, bank contents, selected banks, exemplar lengths, and realized computation. Logs retain bank/key counts, replayed token counts, raw and applied losses, score-gradient summaries, cursor phase, and the exact-zero switch. Model and optimizer state, bank counts and exemplar tokens, replay and data cursors, and request-stream state are checkpointed atomically; the loader validates schema, capacity, scheduler and estimator constants, token types, prompt hashes, key/exemplar consistency, and cursor bounds before accepting a resume.

D.5 FIXED SEMANTIC MAXENT COMPARATOR

Fixed Semantic MaxEnt is an entropy-inspired fresh-rollout comparator. For validator-positive canonical outcomes, an empirical predictor combines successful policy history and successful leave-

one-out peers. If K keys have total count N , unit pseudocounts and one reserved unseen bucket give

$$\hat{p}(a) = \frac{n(a) + 1}{N + K + 1}, \quad \hat{p}(\text{UNSEEN}) = \frac{1}{N + K + 1}. \quad (3)$$

The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}(a_i), C\} - \mathbb{E}_{\hat{p}}[\min\{-\log \hat{p}(a), C\}]}{C}, \quad \eta = .10, C = 5. \quad (4)$$

Invalid and unsuccessful rows receive zero. The centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome present in a fresh rollout, but supplies no direct reinforcement to an absent remembered exemplar. Its heterogeneous empirical effects therefore remain a comparison with exploration rather than a retention guarantee.

For the true conditional distribution $q_\theta = p_\theta(\cdot \mid \mathcal{C})$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (5)$$

This motivates the direction of redistribution, not an unbiased-gradient claim for the empirical estimator. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{A \sim p} [\mathbf{1}\{A \in \mathcal{C}\} (s_A - b) \nabla_z \log p_A] = P \mathbb{E}_{c \sim q} [s_c \nabla_z \log q_c] + (\mathbb{E}_q[s] - b) \nabla_z P.$$

A historical baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla H(q)$. For a single correct key, conditional entropy is zero, yet the implemented unseen bucket can give that key negative semantic advantage. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

E EMPIRICAL RESULTS AND COMPARISONS

E.1 COMPARISON REGISTRY

The primary comparisons cross fresh objective (Dr.GRPO or MaxRL) with verified replay, first across three model scales on Level 1 and then at Qwen2.5-0.5B on Level 2. Table 4 separates these comparisons from the supporting controls and replay-weighting ablation. Terminal populations use the September 11 frozen census. The registered Qwen2.5-0.5B weighting aggregate retains its original frozen population; its endpoint values are unchanged in that census.

Table 4: **Comparison scope and available terminal evidence.** A complete block has five registered paired seeds. Primary partial intersections retain their exact counts and receive no interval. UCPO and sparse RLEP-Dr are reported only at the two smaller model scales; their denominators cover all five benchmark domains and five registered seeds at each selected scale.

Comparison	Scope	Terminal coverage	Question
Dr.GRPO / ReplayDr.GRPO	Level 1, all three scales	74/75 pairs; 14/15 complete blocks	Verified memory beyond fresh reward
MaxRL / ReplayMaxRL	Level 1, all three scales	67/75 pairs; 12/15 complete blocks	Memory beyond a stronger fresh objective
Four-method factorial	Level 2, Qwen-0.5B	4/5 complete domains; two additional Pantry Dr.GRPO pairs	Transfer to harder matched tasks
GRPO	Level 1, all three scales	75/75 runs	Reward-standardization control
UCPO	Level 1, 0.5B and 1B	50/50 runs	On-policy advantage redistribution
Sparse RLEP-Dr	Level 1, 0.5B and 1B	47/50 runs	Success memory without uniform key balancing
Fixed Semantic MaxEnt	Level 1, registered scale/domain blocks	10/15 complete blocks	Fresh-rollout semantic exploration
Uniform / frequency replay	Level 1, nine registered blocks	43/45 treatment cells; 7/9 complete blocks	Contribution of replay-key balancing

E.2 TERMINAL COMPARISONS ACROSS SCALES

Within each model–domain–seed comparison, methods share initialization, data identities, the eight-pass horizon, evaluation requests, verifier, and replay scheduling code. The replay control runs the same bank and scoring path with zero replay derivative. Resulting policies can build different banks. The admissible intersections are defined in Appendix J.1.

Table 5: **Matched results after eight training passes.** Columns compare Dr.GRPO with ReplayDr.GRPO after exactly eight training passes. Entries are means over the displayed paired-seed intersection; n @pass gives its size and training pass. P, M, and D denote `pass@8`, `mean@8`, and `distinct@8`. All source-admissible terminal pairs are included: Falcon Countdown has $n = 4$, other blocks $n = 5$ (Appendix J.1). No missing endpoint is imputed. Pale green marks a ReplayDr.GRPO mean that is strictly higher than its matched Dr.GRPO mean.

Model	Domain	n @pass	Dr.GRPO			ReplayDr.GRPO		
			P	M	D	P	M	D
Qwen2.5-0.5B	Graph coloring	5@8.0	.323	.323	.325	.969	.404	2.438
	Countdown	5@8.0	.480	.465	.487	.672	.597	1.637
	Python factors	5@8.0	.172	.172	.172	.528	.507	.571
	MathIR	5@8.0	.512	.464	.513	.796	.688	.829
	PantryPlan	5@8.0	.522	.522	.522	.729	.545	1.539
Falcon3-1B	Graph coloring	5@8.0	.656	.300	1.034	.809	.334	1.717
	Countdown	4@8.0	.325	.267	.415	.632	.343	.838
	Python factors	5@8.0	.000	.000	.000	.979	.722	1.107
	MathIR	5@8.0	.095	.068	.097	.736	.292	.756
	PantryPlan	5@8.0	.633	.633	.633	.751	.560	1.576
Qwen2.5-3B	Graph coloring	5@8.0	.682	.360	1.060	.836	.349	1.637
	Countdown	5@8.0	.609	.569	.628	.746	.505	.962
	Python factors	5@8.0	.069	.069	.069	.659	.603	.852
	MathIR	5@8.0	.278	.168	.279	.686	.580	.706
	PantryPlan	5@8.0	.638	.558	.841	.744	.546	1.514

Dr.GRPO retention contrasts. Both co-primary block means are positive in all 14 complete five-seed comparisons, and raw `distinct@8` increases in all 74 admissible pairs. At Qwen2.5-0.5B, Countdown `pass@8` rises from .480 to .672 and raw `distinct@8` from .487 to 1.637; the extra-mode effect is +.959 (paired 95% interval [+.863, +1.054]). Python correctness rises from .172 to .528, with an extra-mode effect of +.043 ([−.077, +.164]); MathIR rises from .512 to .796, with extra modes +.030 ([+.001, +.060]). These intervals are nominal, unadjusted Student- t estimates over five paired training seeds. Extra modes remain coupled to correctness. Countdown keys identify computation routes, but these held-out endpoints do not identify which previously discovered routes survived training.

Figure 8 resolves the domain-specific results behind the main cross-domain display. The 3B MaxRL pairs use Graph/Countdown/Python/MathIR/Pantry counts 5/3/5/3/1; the Dr.GRPO pair uses five seeds except Falcon Countdown, which uses four. These populations remain separate: a factorial interaction requires the common four-arm intersection, not differences of aggregates with different seeds.

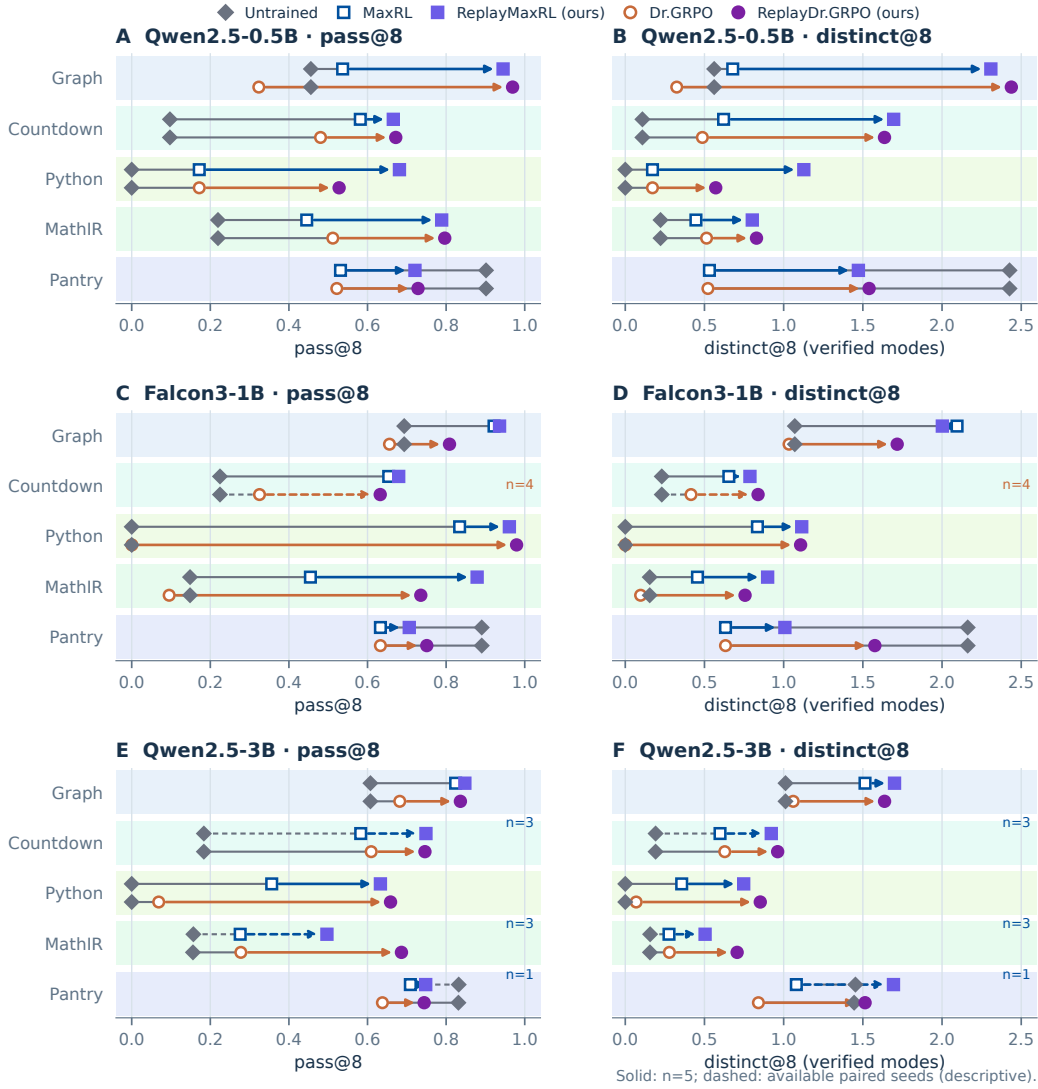


Figure 8: **Per-domain results behind the cross-domain averages in Figure 5.** Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show correctness and raw support. MaxRL–ReplayMaxRL pairs retain five terminal seeds throughout the two smaller models. Qwen2.5-3B uses $n = 5, 3, 5, 3, 1$ in domain order; dashed connectors mark partial blocks. Dr.GRPO–ReplayDr.GRPO uses four admissible seeds for Falcon Countdown and five elsewhere. Domain tints match Figure 2. Replay arrows, trained-arm markers, and untrained references use the same encoding as the main figure. The E118 model index groups the archived exports by model and domain.

Write $P = \text{pass@8}$, $D = \text{distinct@8}$, and $B = D - P$. The derived B counts verified modes beyond the first correct outcome in each sampled set and remains coupled to correctness. The endpoint tables report exact paired terminal effects. A dash denotes no observed pair; partial means retain $n/5$ without an interval. Complete-block intervals are nominal 95% estimates, without multiple-comparison or repeated-look adjustment. All underlying arm means and seed effects remain in the adjacent machine-readable records.

Table 6: **ReplayMaxRL effects across model scales at pass 8.** All effects are ReplayMaxRL minus MaxRL on the displayed paired seeds. Brackets give paired Student- t intervals for ΔB in complete blocks. The machine-readable report also retains both arm means, all per-seed effects, and intervals for P and D .

Model	Domain	Replay arm	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	ReplayMaxRL	5/5	+0.408	+1.630	+1.221 [+0.926, +1.517]
Qwen-0.5B	Countdown	ReplayMaxRL	5/5	+0.084	+1.074	+0.990 [+0.859, +1.121]
Qwen-0.5B	Python	ReplayMaxRL	5/5	+0.509	+0.955	+0.446 [-0.156, +1.048]
Qwen-0.5B	MathIR	ReplayMaxRL	5/5	+0.343	+0.356	+0.013 [+0.008, +0.017]
Qwen-0.5B	PantryPlan	ReplayMaxRL	5/5	+0.189	+0.941	+0.751 [+0.520, +0.982]
Falcon-1B	Graph	ReplayMaxRL	5/5	+0.014	-0.093	-0.107 [-0.220, +0.005]
Falcon-1B	Countdown	ReplayMaxRL	5/5	+0.025	+0.133	+0.108 [+0.082, +0.134]
Falcon-1B	Python	ReplayMaxRL	5/5	+0.127	+0.280	+0.153 [-0.272, +0.578]
Falcon-1B	MathIR	ReplayMaxRL	5/5	+0.424	+0.443	+0.019 [+0.006, +0.031]
Falcon-1B	PantryPlan	ReplayMaxRL	5/5	+0.073	+0.375	+0.302 [+0.034, +0.569]
Qwen-3B	Graph	ReplayMaxRL	5/5	+0.023	+0.186	+0.163 [+0.021, +0.305]
Qwen-3B	Countdown	ReplayMaxRL	3/5	+0.166	+0.324	+0.158
Qwen-3B	Python	ReplayMaxRL	5/5	+0.277	+0.391	+0.115 [+0.080, +0.149]
Qwen-3B	MathIR	ReplayMaxRL	3/5	+0.221	+0.228	+0.007
Qwen-3B	PantryPlan	ReplayMaxRL	1/5	+0.039	+0.613	+0.574

At Qwen2.5-0.5B, ReplayMaxRL adds 1.221, .990, and .751 extra modes on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph instead has a negative mean raw-support effect (-0.093, interval [-0.229, +0.043]), and its correctness effect is inconclusive (+0.014, [-0.016, +0.044]). Positive cross-domain effects therefore do not imply a breadth gain in every domain. The complete Qwen2.5-3B Graph and Python blocks have positive extra-mode intervals while their correctness intervals include zero. The three partial 3B domains retain their own seed sets; the main figure’s equally weighted domain average is descriptive.

Cross-domain factorial summaries. Averaging five domains equally within each paired seed, ReplayMaxRL minus MaxRL gives +.307 on `pass@8` and +.991 on `raw distinct@8` at Qwen2.5-0.5B (unadjusted 95% intervals [+0.208, +0.406] and [+0.815, +1.167]), and +.133/+0.228 at Falcon3-1B ([+0.043, +0.223] and [+0.094, +0.361]). Both use five paired seeds. The Dr.GRPO replay effects on these two endpoints are +.337/+0.999 at Qwen2.5-0.5B, +.442/+0.774 at Falcon3-1B, and +.279/+0.559 at Qwen2.5-3B. The Falcon Dr.GRPO average uses four common admissible seeds without an interval; the other two use five. These are post-hoc descriptive cross-domain summaries. At Qwen2.5-3B, MaxRL/ReplayMaxRL arm means are .550/.695 on correctness and .765/1.113 on raw support. This track weights the five domain-specific paired means equally, with Graph/Countdown/Python/MathIR/Pantry counts 5/3/5/3/1; it has neither a common cross-domain seed trajectory nor an interval. Graph and Python use seeds 70–74; Countdown and MathIR use 70, 71, and 74; Pantry uses 72. The common four-arm intersection contains eleven complete five-seed blocks and Falcon Countdown at four seeds. Factorial interactions require that intersection rather than differences between aggregates using different seeds.

E.3 TERMINAL COMPARISON ON THE HARDER LEVEL

Figure 6 uses Graph, Countdown, Python, and MathIR: each has all five seeds at step 3,072 for both levels and all four methods. Means average those four domains equally within each seed, then the five seed means. This is a descriptive 20-cell comparison, not a five-domain estimate; the two levels have separate held-out prompts. At Level 2, PantryPlan has three Dr.GRPO, three ReplayDr.GRPO, one MaxRL, and one ReplayMaxRL terminal endpoint. Its only terminal pairs are Dr.GRPO/replay seeds 43 and 46, with no common four-arm seed. Independently available arm means do not estimate paired effects.

Table 7: **Replay effects on the harder Level-2 tasks at pass 8.** MaxRL and Dr.GRPO identify the fresh objective; each effect is its replay variant minus the corresponding objective without replay. Each row uses its own pairwise intersection, which can exceed the four-method intersection. Brackets give paired Student- t intervals for ΔB only when all five registered pairs are available.

Model	Domain	Objective	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Dr.GRPO	5/5	+0.561	+1.019	+0.458 [+0.394, +0.522]
Qwen-0.5B	Graph	MaxRL	5/5	+0.305	+0.668	+0.363 [+0.280, +0.446]
Qwen-0.5B	Countdown	Dr.GRPO	5/5	+0.025	+0.045	+0.020 [-0.014, +0.055]
Qwen-0.5B	Countdown	MaxRL	5/5	+0.040	+0.057	+0.017 [+0.001, +0.032]
Qwen-0.5B	Python	Dr.GRPO	5/5	+0.075	+0.400	+0.325 [-0.228, +0.878]
Qwen-0.5B	Python	MaxRL	5/5	+0.037	+0.212	+0.175 [-0.295, +0.645]
Qwen-0.5B	MathIR	Dr.GRPO	5/5	+0.493	+0.493	+0.000 [+0.000, +0.000]
Qwen-0.5B	MathIR	MaxRL	5/5	+0.173	+0.175	+0.002 [-0.003, +0.006]
Qwen-0.5B	PantryPlan	Dr.GRPO	2/5	+0.234	+0.367	+0.133
Qwen-0.5B	PantryPlan	MaxRL	0/5	—	—	—

Level-2 Graph provides the clearest replay effects on correctness and extra modes under both objectives. Countdown’s ReplayMaxRL extra-mode effect is small (+.017, interval [+0.001, +0.032]), while its correctness and raw-support intervals include zero. Python and MathIR have positive mean correctness and raw-support effects with intervals including zero; MathIR’s extra-mode effects are near zero. Pantry’s two-seed Dr.GRPO replay contrast is +.234 on P , +.367 on D , and +.133 on B , without an interval.

For each common seed and endpoint Y , the replay interaction is

$$I = (Y^{\text{ReplayMaxRL}} - Y^{\text{MaxRL}}) - (Y^{\text{ReplayDr.GRPO}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises P relative to Dr.GRPO by +.233 ([+.140, +.325]), while the replay interaction on P is -.256 ([-.329, -.183]). This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 8: **Fresh-objective effects and replay interactions on Level 2.** All three contrasts within a domain use the same four-arm seed intersection. The interaction is (ReplayMaxRL - MaxRL) - (ReplayDr.GRPO - Dr.GRPO). Each entry gives its mean and, only for a complete five-seed block, a paired unadjusted 95% Student- t interval. Archived four-arm exports: E119 model index.

Domain	n	Contrast	ΔP	ΔD	ΔB
Graph	5	MaxRL - Dr.GRPO	+.233 [+0.140, +0.325]	+.322 [+0.162, +0.482]	+.089 [+0.019, +0.159]
Graph	5	ReplayMaxRL - ReplayDr.GRPO	-.023 [-0.062, +0.016]	-.029 [-0.198, +0.140]	-.006 [-0.142, +0.130]
Graph	5	Replay $\times$ MaxRL interaction	-.256 [-0.329, -.183]	-.351 [-0.446, -.255]	-.095 [-0.186, -.003]
Countdown	5	MaxRL - Dr.GRPO	-.014 [-0.073, +0.045]	-.014 [-0.073, +0.045]	+0.000 [+0.000, +0.000]
Countdown	5	ReplayMaxRL - ReplayDr.GRPO	+0.002 [-0.008, +0.012]	-.002 [-0.047, +0.044]	-.004 [-0.042, +0.035]
Countdown	5	Replay $\times$ MaxRL interaction	+0.016 [-0.047, +0.078]	+0.012 [-0.068, +0.092]	-.004 [-0.042, +0.035]
Python	5	MaxRL - Dr.GRPO	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]
Python	5	ReplayMaxRL - ReplayDr.GRPO	-.037 [-0.232, +0.157]	-.188 [-0.988, +0.613]	-.150 [-1.014, +0.713]
Python	5	Replay $\times$ MaxRL interaction	-.037 [-0.232, +0.157]	-.188 [-0.988, +0.613]	-.150 [-1.014, +0.713]
MathIR	5	MaxRL - Dr.GRPO	+.283 [-0.193, +0.759]	+.283 [-0.193, +0.759]	+0.000 [+0.000, +0.000]
MathIR	5	ReplayMaxRL - ReplayDr.GRPO	-.036 [-0.158, +0.085]	-.035 [-0.158, +0.088]	+0.002 [-0.003, +0.006]
MathIR	5	Replay $\times$ MaxRL interaction	-.320 [-0.864, +0.225]	-.318 [-0.860, +0.224]	+0.002 [-0.003, +0.006]
PantryPlan	0	MaxRL - Dr.GRPO	—	—	—
PantryPlan	0	ReplayMaxRL - ReplayDr.GRPO	—	—	—
PantryPlan	0	Replay $\times$ MaxRL interaction	—	—	—

Level-2 Graph endpoint detail. The matched construction lowers frozen-model pass@8 from .438 to .148. At pass 8, Dr.GRPO/MaxRL reach .202/.434, versus .763/.739 with their replay variants. The paired correctness gains are +.561 and +.305 (unadjusted 95% Student- t intervals [+0.445, +0.677] and [+0.250, +0.360], five seeds each). The weakest replay run (.711) exceeds the strongest non-replay run (.477). Replay adds 1.019/.668 raw distinct@8, yielding 1.243/1.214 under the two fresh objectives. These observations show improved correctness and sampled support

on harder Graph; they do not establish that broader support causes the correctness gain. Cross-level prompts are distinct, and the four-domain Level-2 aggregate excludes the incomplete Pantry factorial.

E.4 ACCURACY AND VERIFIED MODES THROUGHOUT TRAINING

The following curves include every primary method and model scale used in the main paper. Each objective pair uses its domain-specific terminal seed intersection as one fixed cohort throughout training. A checkpoint is plotted only when all members have complete, valid evaluations for both methods. Missing or conflicting checkpoints leave gaps; seed counts never shrink to extend a mean. Four fixed-seed draws of eight samples on the same 128 held-out prompts define each evaluation. Accuracy and mode curves use identical seeds and checkpoints, without smoothing or outcome-based selection.

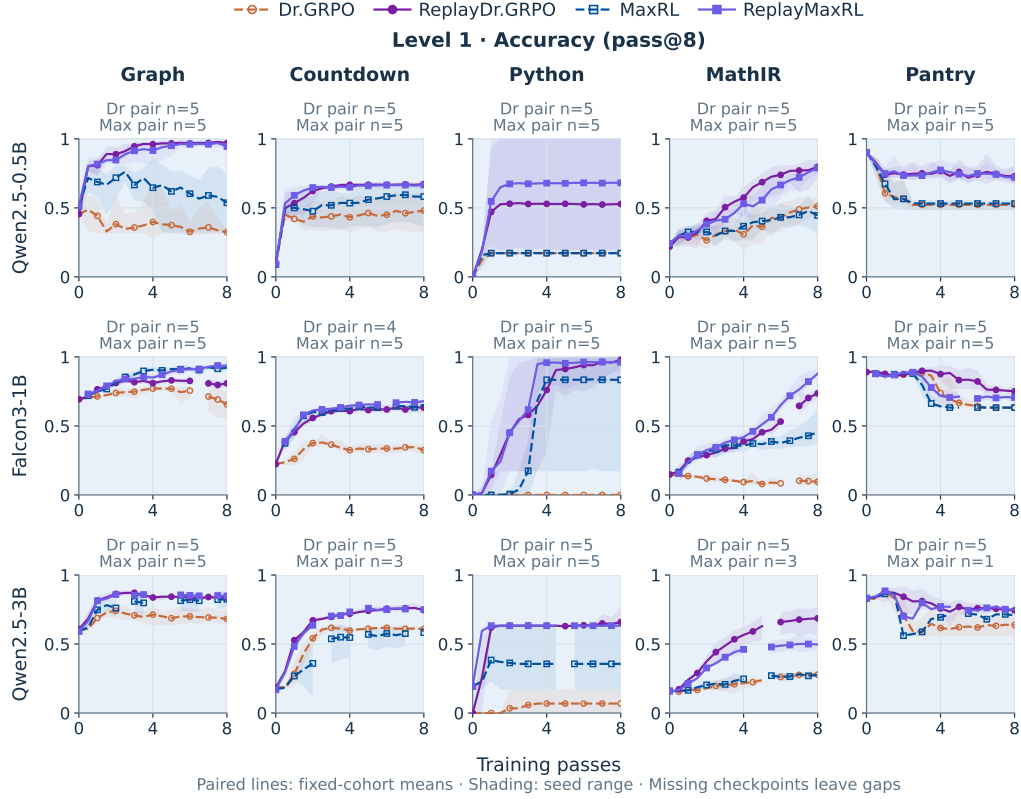


Figure 9: **Accuracy throughout training at all three model scales.** Rows are Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns are the five domains. Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL use the same method colors as Figure 5. Lines are fixed-cohort means of pass@8; bands are seed ranges, not confidence intervals. Counts identify each objective’s terminal paired seeds. Qwen2.5-3B MaxRL retains Graph/Countdown/Python/MathIR/Pantry counts 5/3/5/3/1; Falcon Countdown Dr.GRPO/replay uses four. Gaps indicate missing or conflicting checkpoints.

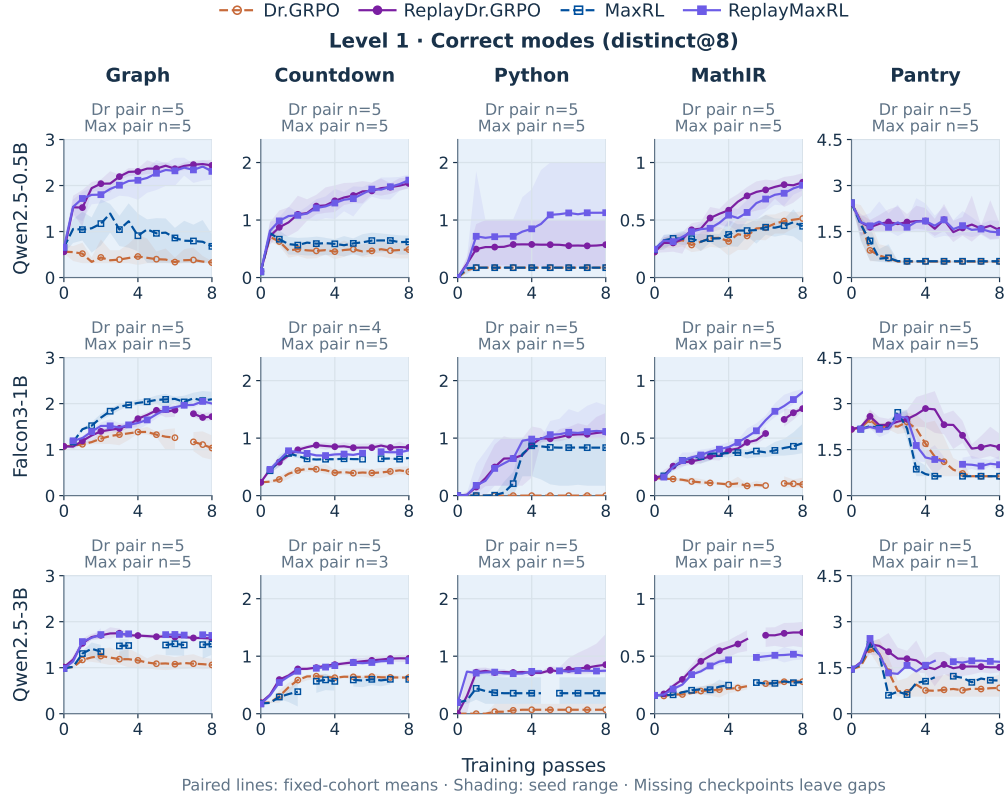


Figure 10: **Verified modes throughout training at all three model scales.** The same model rows, methods, seeds, and checkpoints as Figure 9, now measuring `distinct@8`. Lines are descriptive fixed-cohort means and bands show seed ranges. A count of one has no band. Domain-specific vertical scales are shared across model rows; absolute heights across different domains should be read against their labeled axes. Missing evaluations are never carried forward or joined across a gap.

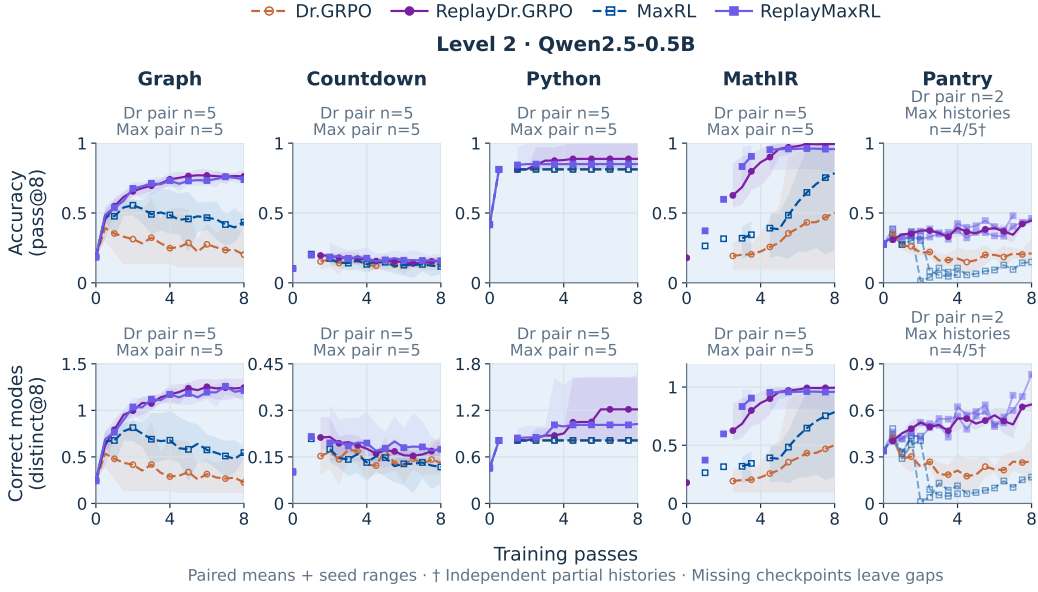


Figure 11: **Accuracy and verified modes during Level-2 training.** Qwen2.5-0.5B is the model tested at this level. Rows show `pass@8` and `distinct@8` for all four primary methods. Graph, Countdown, Python, and MathIR contain five terminal paired seeds per objective. PantryPlan’s Dr.GRPO/replay means use their two terminal paired seeds; additional MaxRL and ReplayMaxRL histories are individual, unpaired observations, marked separately without a mean or band. They do not estimate a paired replay effect. Both metrics follow the same evaluation rules, with gaps at missing checkpoints and no extrapolation to later training passes.

E.5 SUPPORTING COMPARATORS

GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse RLEP-Dr replays verified successes without uniform canonical-key weighting. Fixed Semantic MaxEnt supplies a fresh-rollout exploration incentive (Appendix D.5). These are controls for alternative explanations of the retention results.

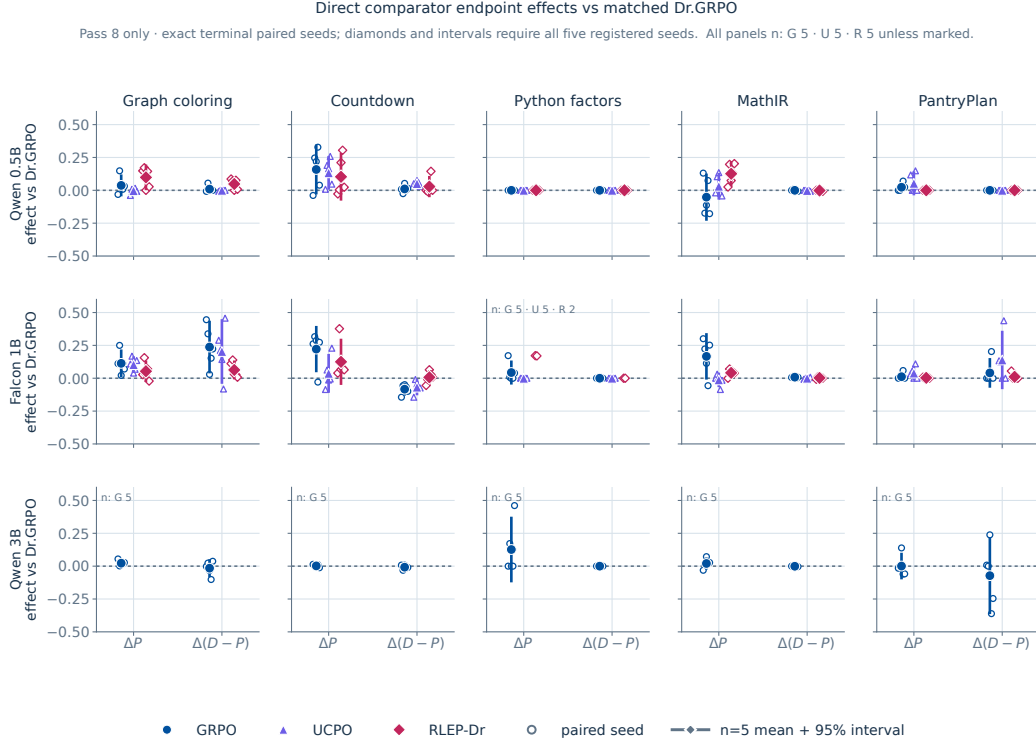


Figure 12: Direct alternatives produce local rather than uniform final gains. Each effect subtracts the same-seed Dr.GRPO control from the comparator after eight passes. Effects are shown for $\text{pass}@8$ and correctness-adjusted breadth. Circles are paired seeds; diamonds and 95% Student- t intervals appear only for complete five-seed blocks. Exact n is printed, and unsupported comparisons remain blank rather than being extrapolated or imputed at any seed.

The alternatives give local rather than uniform benefits. Qwen-0.5B Countdown UCPO raises P by $+.133$ ($[+.005, +.262]$) and B by $+.051$ ($[+.031, +.071]$), whereas its MathIR effects are inconclusive. MathIR RLEP-Dr raises P by $+.126$ ($[+.030, +.222]$) with neutral extra modes ($-.002$, $[-.006, +.003]$); its Countdown effects and Falcon Pantry effects are inconclusive. The full five-seed 3B plain-GRPO blocks likewise have intervals including zero on both correctness and extra modes. RLEP’s frozen offline pool was collected at temperature $.7$, top- $p = .95$; all reported evaluations use the common temperature-1, top- $p = 1$ contract. An offline pool versus an online bank remains an algorithmic difference.

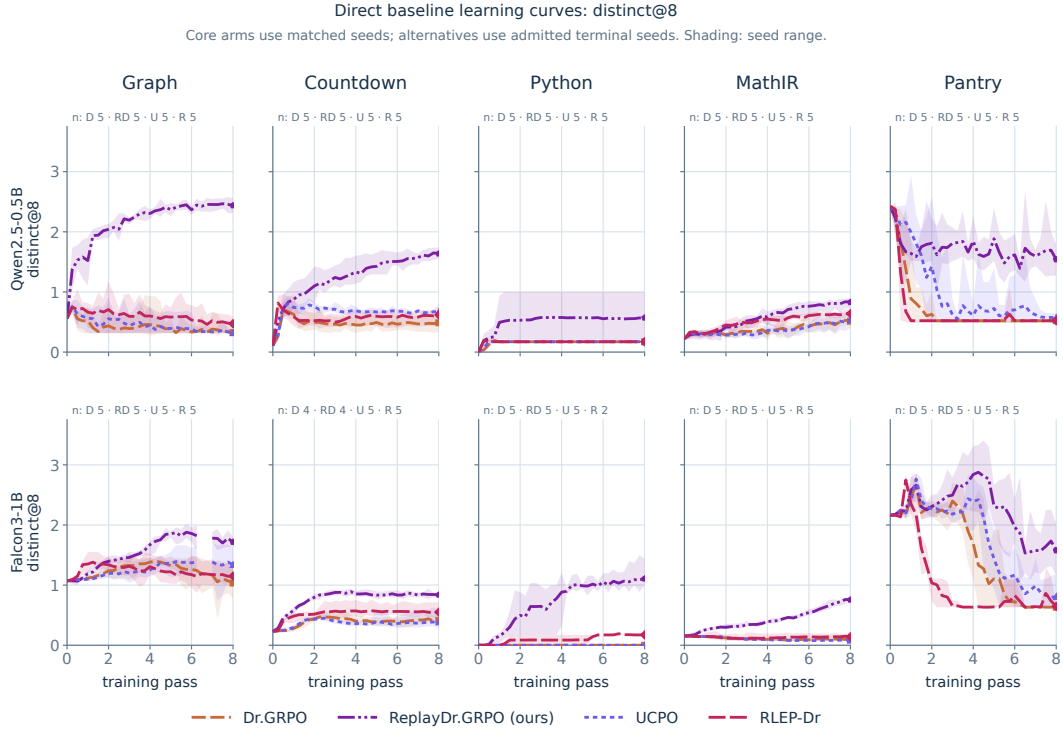


Figure 13: **Verified-mode trajectories for the supporting comparators.** Rows show Qwen2.5-0.5B and Falcon3-1B; columns are the five domains. UCPO and sparse RLEP-Dr are compared with Dr.GRPO and ReplayDr.GRPO at these two smaller scales. Labels give exact method-specific seed counts. Lines are descriptive distinct@8 means and bands are seed ranges. The corresponding accuracy curves use identical populations in Figure 14.

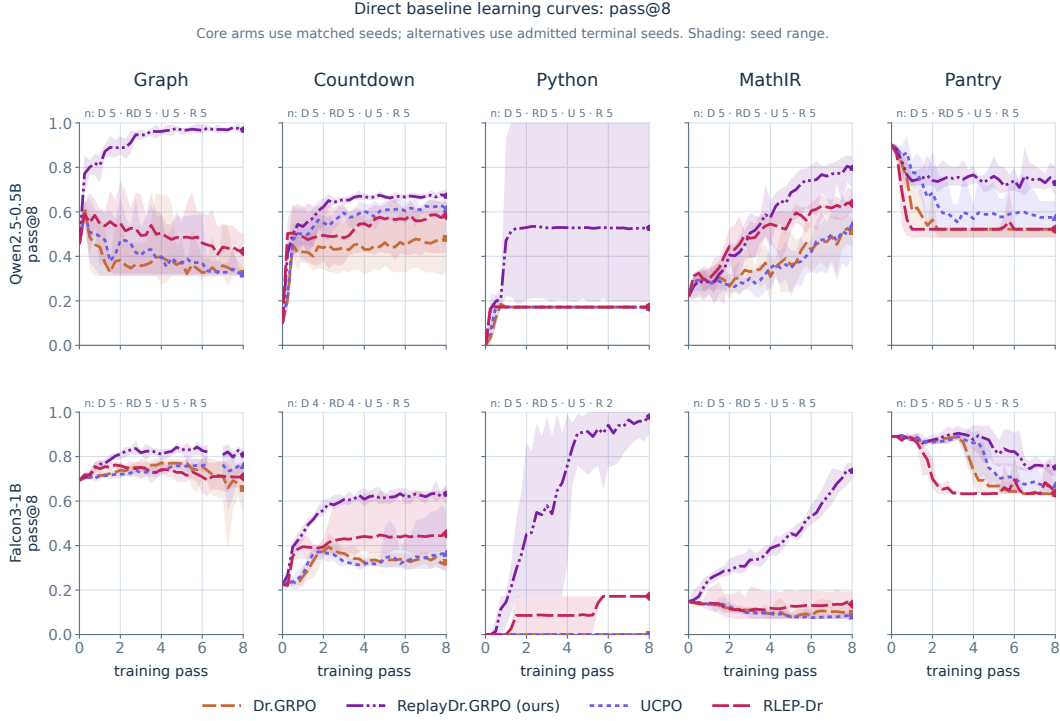


Figure 14: **Accuracy trajectories for the supporting comparators.** The same model rows, domains, methods, terminal seeds, and registered checkpoints as Figure 13, now measuring pass@8. Bands show the seed range and the vertical axes span $[0, 1]$ throughout. MaxRL and ReplayMaxRL at all three scales are compared on both metrics in Figures 9 and 10.

E.6 UNIFORM VERSUS FRESH-FREQUENCY REPLAY

The weighting ablation retains the same bank construction and update code, changing only the within-bank target: uniform weights over retained keys versus weights proportional to validator-positive fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below favor uniform weighting. The registered co-primary contrasts are ΔB and ΔP ; the five-domain Qwen2.5-0.5B aggregate first averages domain effects within each matched seed, giving equal domain weight.

Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired training seeds and use linearly interpolated 2.5th and 97.5th percentiles. A resampled seed carries its complete domain vector when estimating the five-domain aggregate. The endpoints and aggregate were preregistered; the exact seed-resampling and percentile conventions are analysis choices. These nominal intervals quantify seed variability within the fixed benchmark, without resampling prompts or domains or adjusting across comparisons.

Table 9: **Uniform versus fresh-frequency replay at pass 8.** Effects are uniform minus frequency weighting. Brackets give the paired percentile bootstrap interval for ΔB over all 5^5 ordered resamples in complete blocks. Every observed treatment endpoint in this snapshot passes the persisted replay-weight telemetry audit. Partial blocks retain exact seed counts. Paired model exports.

Model	Domain	Weighting	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Uniform	5/5	+0.066	+0.662	+0.596 [+0.513, +0.687]
Qwen-0.5B	Countdown	Uniform	5/5	+0.022	+0.661	+0.639 [+0.544, +0.733]
Qwen-0.5B	Python	Uniform	5/5	-0.077	-0.040	+0.037 [-0.012, +0.123]
Qwen-0.5B	MathIR	Uniform	5/5	-0.005	+0.014	+0.019 [-0.001, +0.038]
Qwen-0.5B	PantryPlan	Uniform	5/5	+0.043	+0.341	+0.298 [+0.174, +0.400]
Falcon-1B	Graph	Uniform	5/5	+0.045	+0.278	+0.233 [+0.182, +0.320]
Falcon-1B	PantryPlan	Uniform	5/5	-0.021	-0.167	-0.146 [-0.564, +0.414]
Qwen-3B	Graph	Uniform	4/5	-0.006	+0.063	+0.070
Qwen-3B	PantryPlan	Uniform	4/5	+0.011	+0.232	+0.222

The registered Qwen2.5-0.5B five-domain effect is +.318 extra modes per prompt ([+.272, +.358]), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The correctness effect is +.010 ([-.119, +.139]), establishing neither a gain nor equivalence. Python’s correctness effects range from -.781 to +.828 across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes (+.233, [+0.182, +0.320]), but its mean sampled correctness decreases by -.0152 ([-.0214, -.0090]); its P interval includes zero. Falcon Pantry is inconclusive: $\Delta B = -.146$ [-.564, +.414] and $\Delta P = -.021$ [-.063, +.026]. Qwen2.5-3B Graph and Pantry have descriptive extra-mode effects of +.070 and +.222 at four pairs each. All observed treatment endpoints pass the persisted replay-weight telemetry audit. The evidence supports a contribution from key balancing in the stated domains, not a uniform accuracy or breadth advantage across every model–domain block.

Evidence boundary. The primary effects are within-domain, within-scale paired comparisons. They do not establish a scaling law, a positive factorial interaction, or preservation of every previously discovered mode. Partial domains are reported as observed and do not support complete-block generality.

F COLLAPSE AND REPLAY MECHANISM CHECKS

F.1 BREADTH BEFORE AND AFTER VERIFIER-ONLY TRAINING

The problem precheck evaluates unmodified Dr.GRPO and GRPO at passes 0 and 8 on all three models and five domains: 150 complete runs, each with four fixed $K = 8$ draws. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. Correctness falls in 10 of the 30 domain–method–scale comparisons, while macro raw `distinct@8` ends above its initial value at Qwen2.5-3B. This distinguishes loss of multiple correct outcomes from the broader question of whether any correct outcome is sampled.

Initial evaluations are measured within each method. Qwen2.5-3B MathIR and Pantry E80-R1 seeds 71–74 carry slightly different initial requests from plain GRPO (.154 versus .162, and .832 versus .828 on P). Those cells therefore have separate initial references.

Graph and PantryPlan contain substantial initial breadth at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-breadth percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. ReplayDr.GRPO does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four admissible seeds. These observations concern sampled held-out support, not the survival histories of particular stored modes.

Uniform-action references are meaningful for the enumerable Graph and Pantry representations. They are not defined for open-ended programs or derivations. Pantry’s bit-uniform

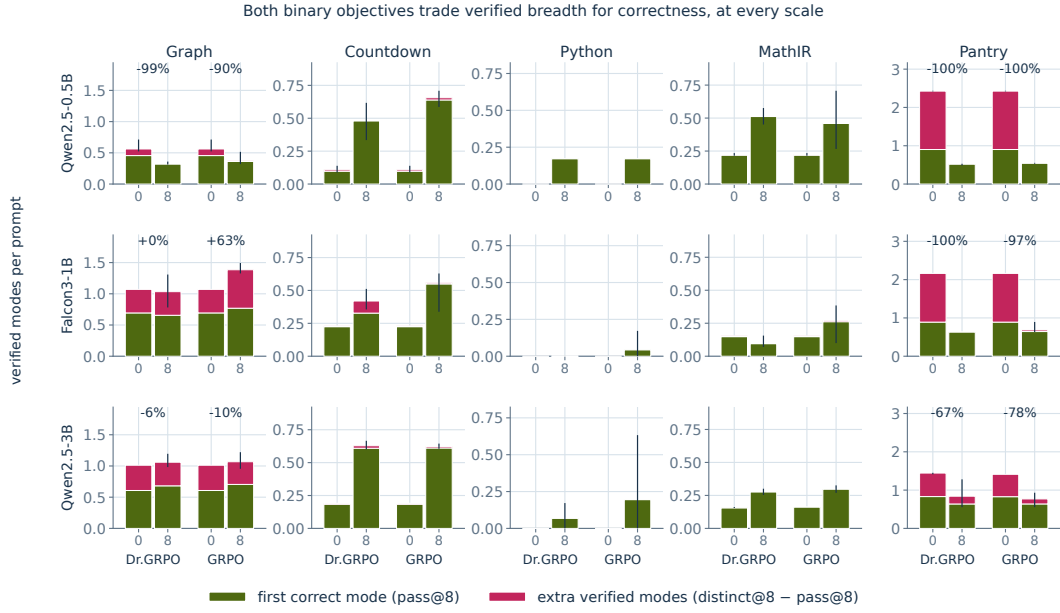


Figure 15: **Both verifier-only objectives reduce verified solution breadth, at every scale.** Bars split distinct@8 into the first correct mode and every verified mode beyond it, at training pass 0 and pass 8, for unmodified Dr.GRPO then GRPO in each panel. All 150 evaluations are complete at five seeds and four fixed $K = 8$ draws; whiskers are the seed range and pass 0 is measured per method. Badges give the change in extra verified modes, printed only where pass-0 breadth exceeds .05 of them; the other three domains start at .011 or less.

Table 10: **Problem precheck, five-domain macro-averages.** Both verifier-only objectives at pass 0 and pass 8, all three scales, five seeds and four fixed $K = 8$ draws per model-domain-method comparison. “Extra modes” is $\text{distinct@8} - \text{pass@8}$; “removed” is the fraction of that quantity lost between pass 0 and pass 8. Breadth loss is present at every scale and is far from uniform across scales, and at Falcon3-1B Dr.GRPO macro pass@8 falls as well.

Model	Dr.GRPO			GRPO		
	pass@8	extra modes	removed	pass@8	extra modes	removed
Qwen2.5-0.5B	0.335 → 0.402	0.329 → 0.002	99.4%	0.335 → 0.435	0.329 → 0.006	98.2%
Falcon3-1B	0.391 → 0.342	0.332 → 0.094	71.6%	0.391 → 0.454	0.332 → 0.135	59.4%
Qwen2.5-3B	0.356 → 0.455	0.205 → 0.120	41.5%	0.356 → 0.490	0.199 → 0.101	49.4%

reference exceeds every final model, so this domain is a support-concentration stress test rather than evidence of complete support coverage.

F.2 FRESH-GRADIENT DEGENERACY AND OBSERVED OUTPUT COLLAPSE

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Domain	Dr.GRPO	ReplayDr.GRPO
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Table 11: **Absolute references for raw distinct@8.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Frozen is the matched pass-0 model; the final two columns are pass-8 means. Dashes avoid pretending that an open-ended program or derivation space has a canonical uniform distribution.

Model	Domain	Uniform	Frozen	Dr.GRPO	ReplayDr.GRPO
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

A raw-response audit of all five Qwen2.5-0.5B Python control seeds establishes observed output collapse directly. At pass 0, a draw has 7.99 unique raw completions per prompt on average. By steps 288–480, every seed produces 32 byte-identical temperature-1 samples per prompt, also identical to the logged greedy completion, on all 128 prompts. This persists through step 3,072. Terminal draws contain 1.00 unique completion per prompt in the control and 1.97 under replay. A collapsed completion can be incorrect: this is one observed completion per prompt, not one correct key everywhere. Aggregate endpoint equality or zero draw spread alone would not establish that conclusion, nor does this finite audit rule out low-probability tails.

These checks explain a vulnerability of the registered sparse binary-reward recipe. They do not establish dominance over methods that keep fresh on-policy signal active. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider-decoding evaluations remain relevant controls beyond the registered factorial.

F.3 FIXED-BANK EXEMPLAR SCORE TRAJECTORIES

The fixed-bank study tracks Qwen2.5-0.5B ReplayDr.GRPO on Graph for seeds 43–47. Before step 384’s fresh group, bank membership and fresh counts are frozen; admission then stops while uniform replay continues through 3,072 updates under the same training recipe. Every frozen exemplar is identified by prompt and canonical-key fingerprints, without selection by its score trajectory. Each scheduled replay visit records mean token log probability s and sequence log probability ℓ . Changes compare an identity’s first and final post-freeze visits; observation steps can differ across identities.

All 3,406 frozen identities across 1,575 seed–prompt pairs have finite, aligned scores at every scheduled visit: 8–9 visits per identity and 29,041 observations in total. The pooled median change is +.038 nat/token (95% interval [+ .025, +.048]), but the 10th percentile is −.256 and 2.61% of identities lose more than .5 nat/token ([1.54, 3.77]%). The worst intermediate change is −1.328 nat/token. On the sequence scale, the median is +.154 nat/sequence ([+.104, +.198]), while 21.99% lose more than .5 nat/sequence. Lengths range from 4 to 100 tokens, so the two threshold percentages have different meanings.

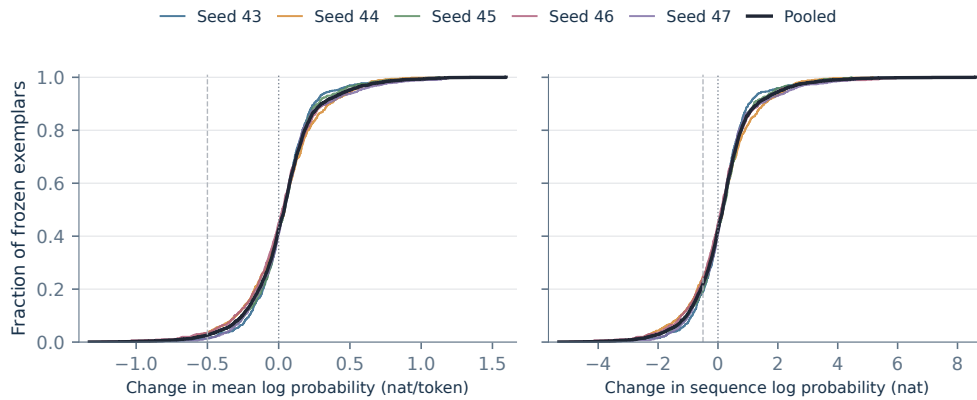


Figure 16: **Fixed-exemplar scores improve centrally but retain a declining tail.** Empirical cumulative distributions show first-to-final post-freeze score changes for every frozen identity, with one curve per seed and the pooled identity distribution. Mean token and sequence scores are separate panels; positive values indicate increased teacher-forced likelihood. Neither panel measures the exact probability of generating a canonical mode during policy sampling.

Pooled summaries weight identities equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap draws (random seed 121), resampling prompts within each original seed and then five seeds with replacement; repeated seeds reuse their within-draw prompt sample. Linear empirical quantiles and percentile intervals are analysis choices. The artifact `paper/results/e121_fixed_bank_survival.json` retains all seed summaries, identity trajectories, source hashes, and audit diagnostics.

These are teacher-forced score surrogates, not exact probabilities of sampling canonical modes. Visits do not cover every intervening update, and the declining tails exclude a claim that every exemplar improves. Without a matched no-replay fixed-bank arm, this study does not identify a causal replay effect. It neither numerically validates Theorem I.6’s probability floor nor certifies a finite-network guarantee. It adds longitudinal measurements of fixed exemplars to the held-out support evidence.

G INFERENCE-ONLY CONCENTRATION IN A HOSTED MODEL

This exploratory evaluation measures the current output distribution of GPT-5.6 Sol under the inherited MODEBENCH interface. It is separate from the matched training interventions: it adds no frontier-model training, replay comparison, or controlled model-size contrast. Only the completed, audited deployment enters these tables.

Protocol and provenance. The September 11, 2026 run uses the Azure deployment `gpt-5.6-sol`; saved provider headers identify snapshot `gpt-5.6-sol-2026-07-09`. Each of the five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. The service returns temperature 1.0 and $\text{top-}p$.98; neither setting was overridden. These settings and the hosted natural-language interface differ from the local training evaluation’s token-level grammar constraints.

Level 1 uses the native E118 evaluation splits retained in E124. Level 2 uses the frozen matched r5 construction; Level 3 uses the matched v3 construction admitted by the completed canonical v3 confirmation. Level 3 is an exploratory test extension here, not a third level of the paper’s training intervention. All cells use the multi-answer split. The Level-2/Level-3 support histograms match a separate fixed-reference Level-1 confirmation reserve, which differs from the native Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Exact prompts, dataset identities and hashes, requests, provider-exposed responses, grades, retry receipts, usage and verifier snapshots are retained in the experiment’s evidence directory. All

15,360 intended samples completed, with distinct response identifiers; 107 failed API attempts were retained and recovered by retries. Provider random-number independence and hidden reasoning contents cannot be audited from the exposed responses.

Table 12: **Hosted correctness and concentration under strict grading.** GPT-5.6 Sol, with medium reasoning. Each level weights five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is the percentage of within-prompt correct pairs sharing a mode, pair weighted within each domain. Brackets give pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8 in this paper (called pass@1 in the hosted report). Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise descriptive intervals.

Table 13: **GPT-5.6 Sol under strict executable grading.** Every cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is distinct@8. C is correct-pair collision (%), with 95% prompt-bootstrap interval); U is the uniform-correct-key collision reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Formatting sensitivity. Strict grades remain primary. A secondary typography normalizer was introduced after inspecting the first 15 responses. It fixes a frozen set of presentation conventions, including escaped Python/LaTeX syntax, and reuses the original executable validators without repairing calculations or program logic. It recovers 2,580 additional verified outputs; strict successes and their keys are retained. This is a post hoc diagnostic, not a replacement benchmark. All 3,072 primary Python responses passed an independent serial recheck with no grade

changes. One cold-start timeout in the secondary grading cache was reconciled with repeated serial verification; the original cache and correction audit remain preserved.

Table 14: **Post hoc formatting-normalized diagnostic.** Same samples and metric definitions as Table 13, with fixed presentation normalization before executable verification. The strict table remains primary; normalization was designed after the first 15 responses.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration and level. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized diagnostic gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not a monotonic law within domains. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses particularly clearly: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 correct pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). Graph was selected after the GPT run for this illustration because its correctness is high, support is certified and formatting normalization changes no scores; both tables report all domains.

Interface and claim limits. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Eight draws cannot establish that an unseen mode has zero probability. The evidence supports concentration under the specified prompts and deployment configuration; it identifies neither a training cause nor a causal difficulty or model-size effect.

Reproduction record. The source record is `artifacts/frontier_modebench_gpt56sol_20260911/summary.json`. Its hash, complete cell statistics and run settings are copied into `results/frontier_hosted_20260911.json`; adjacent generated table bodies supply the displayed values. The evidence directory also retains `completion_audit.json`, `prompt_contract_audit.json`, the original requests and responses, and `interpretation_notes.md`. Hosted-model results are separate from the training-checkpoint archive and its matched-seed comparisons.

H COMPARISON ACROSS HOSTED DEPLOYMENTS

Figure 7 compares verified output breadth across hosted deployments. Its main display applies the frozen formatting normalizer to every deployment and uses the complete revised-wording Opus 5 Python cohort at all three levels. Every response enters its accuracy and mode-count metrics; outputs are not selected by correctness or provider outcome. The other cells use their original prompts. This is a descriptive comparison across the stated task formulations, rather than a common-prompt model ranking.

The tables and Graph figure below preserve every original-protocol result, including failed draws and native refusals. The separate Python comparison reports the original and revised conditions under both grading rules. These records expose the effect of the prompt change without replacing the original evidence. Level 3 remains an exploratory test extension; no hosted-model training or replay intervention is performed. Original deployments enter the comparison only with a passing integrity audit and all 15,360 responses.

Completed hosted comparison. The admitted deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro: 7 complete runs, 107,520 responses, and the same 128 held-out prompts per domain and level with eight stateless samples per prompt. Only runs with a passing completion audit and the identical frozen prompt digest enter these tables. No training, model-side tools or conversation history is added.

Provider configurations. GPT-5.6 Sol: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 5: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. GPT-5.4: hosted API, requested reasoning setting medium, native output limit 8192. Grok 4.3: hosted API, requested reasoning setting medium, native output limit 8192. Kimi K3: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 4.8: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. DeepSeek V4 Pro: hosted API, requested reasoning setting medium, native output limit 8192.

Provider effort labels do not establish equal compute, and native token accounting differs. The saved native request and returned usage determine each configuration; provider defaults remain unspecified whenever native responses omit them.

The appendix retains strict executable grades alongside formatting-normalized scores. The main display uses the same frozen formatting rule for every cell and the complete separately collected revised Opus 5 Python condition; all other displayed cells retain the original request wording. The shared typography diagnostic was developed after the first 15 GPT-5.6 Sol responses and frozen before the prospective comparison runs. It retains its post hoc status for GPT-5.6 Sol. Inherited route preferences and the Level-1 PantryPlan interface change limit cross-level interpretation. Countdown has no certified finite-total-support uniform reference. These data establish neither a training cause nor a causal model-size or difficulty effect. All pair-collision intervals are pointwise; overlapping or nonoverlapping cell intervals are not a paired model-difference test.

Collection interruption: DeepSeek V4 Pro. 13 HTTP-200 bodies had no terminal finish reason or visible answer and were preserved as provider protocol failures. A later storage interruption left 192 logged physical request starts without a saved outcome or usage; their outcome and possible billing remain unknown. The audit registers these attempts separately from the 15,360 terminal draws. Only missing terminal draws were recollected; saved terminal responses, including truncations, were retained. Reported token totals are subtotals of known usage, not zero-cost claims for failed or interrupted attempts. Source-bound recovery sidecars preserve the original receipts and the exact interruption accounting.

H.1 A SEPARATE PYTHON PROMPT SENSITIVITY TEST

After the original Opus 5 Python refusals, synthetic tasks were used to select a direct arithmetic-expression formulation with no system message. We then collected eight independent draws for every original Python test task: all 384 prompts across three levels, with unchanged adaptive thinking, medium effort, 8,192 output tokens, allowed operators, public inputs and frozen graders. The

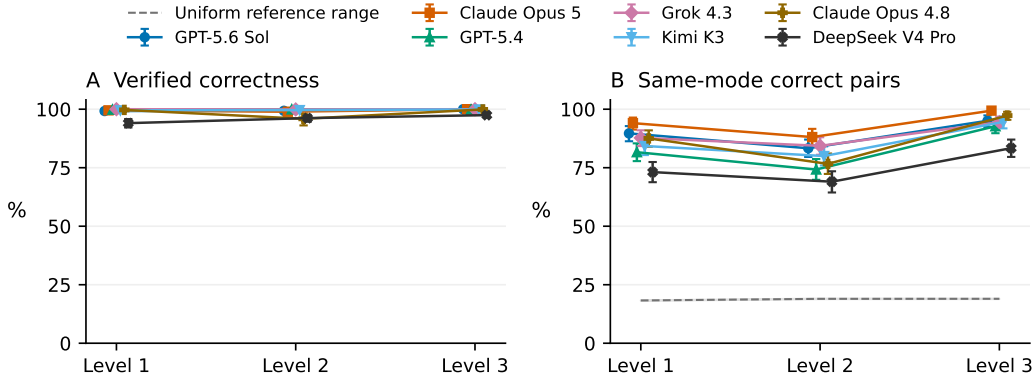


Figure 17: **Verified correctness and concentration in hosted Graph outputs.** Each deployment receives the same 128 prompts per level, with eight stateless samples per prompt. Intervals resample whole prompts. The gray band spans the models’ uniform-correct-key references, conditional on their observed correct draws. Graph was selected after the GPT-5.6 Sol run and before the prospective comparison runs. Level labels do not guarantee increasing difficulty for each model.

revised user instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed input. It does not prescribe a preferred divisor. Both task wording and system-message presence change; collection time also differs. This exploratory follow-up therefore supports a practical prompt repair without isolating its cause. Original refusals and all three residual refusals are retained. The original-protocol appendix tables retain all original Python responses; the main display uses this complete revised condition with an explicit caption note.

Table 15: **Opus 5 Python: original versus plain task wording.** Each row has 1,024 responses. R counts native refusals; A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). Strict and normalized results use the same frozen grader and typography rule. Original Level-2/3 collision is undefined because no prompt has two correct draws.

Condition	L	R	A	Strict		C	Normalized		
				D	D		A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4	
	2	1021	0.3	0.02	–	0.3	0.02	–	
	3	1024	0.0	0.00	–	0.0	0.00	–	
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9	
	2	0	71.8	1.31	84.7	100.0	1.38	85.7	
	3	3	76.2	1.38	82.7	99.7	1.41	84.1	

The new condition has no API failures or token-limit truncations. Strict grading accepts 2,282 responses; the frozen rule rescues 787 more by converting escaped percent signs (\%) to Python modulo (%). Thus every non-refused answer passes after this formatting-only correction. Normalized collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The residual three refusals occur on two prompts whose other identical requests are accepted. No retry-until-accepted selection is used.

The paired analysis resamples whole eight-draw prompt groups across conditions (20,000 replicates). Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2, 100.0] percentage points (pointwise 95% intervals). Output draws themselves are independent, and the different correct-pair populations preclude interpreting a raw collision difference as a matched-population prompt effect.

The saved sensitivity record binds the original and revised native receipts, request payloads, strict regrade audit and frozen normalization outputs.

H.2 NATIVE PROVIDER OUTCOMES AND UNDEFINED CONCENTRATION

Provider-declared refusals and filtering are deployment outcomes, not evidence that the underlying model cannot solve the mathematical task. Empty refusals remain failed draws under the frozen verifier; no requests or grades are overwritten. Classification uses native metadata, not a semantic detector of refusals written as ordinary answer text.

Correct-pair collision is undefined when no prompt supplies two correct draws. The five-domain mean is therefore undefined if any domain has no eligible pairs; such entries remain – rather than zero or a mean over fewer domains.

GPT-5.6 Sol native completion states: completed: 15,360. Empty visible answers are reported separately from provider refusals.

Table 16: **Native service outcomes: GPT-5.6 Sol.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Claude Opus 5 native completion states: end_turn: 12,838, refusal: 2,522. Empty visible answers are reported separately from provider refusals.

Claude Opus 5 returns provider-declared refusals for 409, 1021, 1024 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are cyber: 2454. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

GPT-5.4 native completion states: completed: 15,355, incomplete: 5. Empty visible answers are reported separately from provider refusals.

Grok 4.3 native completion states: stop: 15,360. Empty visible answers are reported separately from provider refusals.

Kimi K3 native completion states: length: 15, stop: 15,345. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 native completion states: end_turn: 15,355, max_tokens: 5. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 returns provider-declared refusals for 0, 0, 0 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are not supplied. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

DeepSeek V4 Pro native completion states: length: 632, stop: 14,728. Empty visible answers are reported separately from provider refusals.

The result record `results/frontier_comparison_20260911.json` binds every source summary, completion audit, frozen prompt digest, raw-sample digest and primary graded sample

Table 17: **Native service outcomes: Claude Opus 5.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	409	0	409
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	1021	0	1021
2	MathIR	35	0	35
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	1024	0	1024
3	MathIR	33	0	32
3	PantryPlan	0	0	0

Table 18: **Native service outcomes: GPT-5.4.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	1
2	Graph	0	0	0
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	1
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	2

Table 19: **Native service outcomes: Grok 4.3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 20: **Native service outcomes: Kimi K3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 21: **Native service outcomes: Claude Opus 4.8.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	3
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	1

Table 22: **Native service outcomes: DeepSeek V4 Pro.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	59
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	4
1	PantryPlan	0	0	13
2	Graph	0	0	38
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	59
2	PantryPlan	0	0	80
3	Graph	0	0	24
3	Countdown	0	0	7
3	Python	0	0	0
3	MathIR	0	0	100
3	PantryPlan	0	0	246

Table 23: **Strict level summaries for completed hosted deployments.** Each level weights the five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is correct-pair collision (% with pointwise 95% whole-prompt intervals), pair weighted within each domain. Provider settings differ; these are descriptive outputs.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	–
Claude Opus 5	3	77.8	79.8	1.07	–
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Table 24: **GPT-5.6 Sol: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Table 25: **Claude Opus 5: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5

Table 26: **GPT-5.4: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–
1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–
2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–
3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 27: **Grok 4.3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 28: **Kimi K3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–
1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–
2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–
3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 29: **Claude Opus 4.8: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5

Table 30: **DeepSeek V4 Pro: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 31: **GPT-5.6 Sol: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Table 32: **Claude Opus 5: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5

Table 33: **GPT-5.4: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 34: **Grok 4.3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 35: **Kimi K3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 36: **Claude Opus 4.8: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5

Table 37: **DeepSeek V4 Pro: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

digest. Pending deployments are omitted from both the figure and numerical comparisons. The figure was selected after the reference GPT run for its high strict correctness, certified support and lack of formatting corrections, and before collecting the prospective comparison runs. All five domains are retained in the tables, including disagreement across models.

I MODE COLLAPSE AND VERIFIED REPLAY

This appendix establishes the categorical claims behind the method: centered binary-reward estimators amplify a pre-existing imbalance among correct modes, whereas persistent verified replay supplies a loss barrier for banked modes. The complete-response argument then states what would be needed to transfer that protection to a language model. These results concern idealized update rules; they do not certify the implemented AdamW/PPO trajectories.

I.1 CATEGORICAL REDUCTION AND MEAN UPDATES

Fix one prompt and a finite response alphabet $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$, with $m \geq 2$ correct execution modes and $n \geq 1$ incorrect categories. Let $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$. Thus P measures correctness and q the distribution among correct modes. Throughout the mean-flow analysis, logits are finite initially, $0 < P(0) < 1$, explicit reference KL is zero, and the prompt is isolated.

Draw a group $A_1, \dots, A_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[A_i \in \mathcal{C}]$, and let $M = \sum_i R_i$. Both GRPO variants can be written, at the on-policy point where the PPO ratio equals one, as

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(M) \left(R_i - \frac{M}{G} \right) \nabla_z \log p_{A_i}. \quad (6)$$

For Dr.GRPO, $w(M) = 1$ after absorbing the shared positive length scale. For binary-reward GRPO, $w(M)$ is the reciprocal group reward standard deviation on mixed groups. We absorb the same fixed response-length normalizer in both variants; response-dependent row normalization is outside this reduction. In either case $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. We define the entire update as zero on all-equal groups, without evaluating an undefined reciprocal standard deviation; likewise, $w(M)M(G-M)$ below is defined as zero when $M = 0$ or $M = G$. We take explicit reference KL to be zero, as in the experiments.

Lemma I.1 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected group update is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(M)M(G-M)]}{G^2 P(1-P)} > 0. \quad (7)$$

For Dr.GRPO, $c_G(P) = (G-1)/G$. Hence reward centering and binary reward standardization change the speed, but not the expected direction, of the binary-reward policy gradient.

Proof. Condition on the complete binary reward vector, whose sum is M . The score identities give

$$\mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 0] = \nabla_z \log(1-P).$$

There are M correct rows with centered reward $(G-M)/G$ and $G-M$ incorrect rows with centered reward $-M/G$. Therefore the conditional expectation of the unaveraged sum in (6) is

$$w(M) \frac{M(G-M)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(M) \frac{M(G-M)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $M \sim \text{Binomial}(G, P)$ yield (7). For $w \equiv 1$, $\mathbb{E}[M(G-M)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1) P^j (1-P)^{G-2-j}.$$

The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. This establishes the smooth, positive extension to $P = 0, 1$ used in the subsequent flow and convergence arguments.

The next lemma specializes the practical-estimator analysis of Tajwar et al. (2026, Section 4.3 and Appendix D, Theorem 5) to our categorical notation; we include the derivation for completeness.

Lemma I.2 (Binary MaxRL has the same correctness-gradient direction). *Let MaxRL assign $A_i^{\text{MaxRL}} = GR_i/M - 1$ when $M > 0$ and zero when $M = 0$, as in Section 3.1. At the on-policy point, its expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1 - P)^{G-1}}{P} > 0, \quad (8)$$

with continuous endpoint values $c_G^{\text{MaxRL}}(0) = G - 1$ and $c_G^{\text{MaxRL}}(1) = 1$.

Proof. Condition on $M = m > 0$. Each successful row has advantage $(G - m)/m$ and each failed row has advantage -1 . Using the same conditional score identities as above, the expected unaveraged sum is

$$(G - m)(\nabla_z \log P - \nabla_z \log(1 - P)) = \frac{G - m}{P(1 - P)} \nabla_z P.$$

After the outer factor $1/G$ and averaging over $M \sim \text{Binomial}(G, P)$,

$$\mathbb{E}[(G - M)\mathbf{1}\{M > 0\}] = G(1 - P) - G \Pr(M = 0) = G[(1 - P) - (1 - P)^G].$$

Substitution gives Equation (8). Its endpoint values follow by continuity (or the finite geometric-series identity), which also gives the stated continuous extension at both boundaries. $\square$

The same finite-series identity gives the potential used below:

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1 - P)^k}{k}. \quad (9)$$

In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, $\text{best}@k$ equals $\text{pass}@k$, so this is a harmonic mixture of sampling objectives for $k = 1, \dots, G - 1$. The upper limit is $G - 1$, not G : the implemented centered estimator drops the entire update on all-failure groups. Subtracting an unconditional score baseline even on those groups instead gives the order- G estimator. This distinction concerns the exact finite-group expectation at the on-policy point; it does not assert unbiasedness of subsequent clipped or adaptive-optimizer updates.

I.2 WINNER-TAKE-ALL COLLAPSE IN THE CATEGORICAL MEAN FLOW

Consider $\dot{z} = c_G(P) \nabla_z P$, with any of the three smooth positive functions established above. This infinitesimal expected-update model removes sampling noise and retains independently trained Euclidean logits. The conditional dynamics reduce to the standard frequency-dependent replicator system with fitness $f_c(q) = q_c$ (Hofbauer & Sigmund, 1998). The result is a specialization to the group estimators, not a new general theorem about replicator dynamics. Related correctness-indifferent collapse analyses include Sinha et al. (2026); Lochab et al. (2026).

Theorem I.3 (Standard replicator specialization for binary-reward RL). *Start from finite logits with $0 < P(0) < 1$. If the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

Thus these GRPO, Dr.GRPO, and binary MaxRL mean flows converge to a single correct execution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward.

Proof. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P) p_c(1 - P), \quad \dot{z}_a = -c_G(P) p_a P.$$

Also $\dot{P} = c_G(P) \|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (10)$$

The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ; normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly uniform correct distribution is therefore a special initial condition. The theorem concerns limiting concentration: finite softmax logits already give positive probabilities at every finite time, so finite-time positivity alone is not a retention result.

Dependence on optimization geometry. For a smooth parameterization $z = z(\theta)$, Euclidean parameter flow instead induces $\dot{z} = c_G(P)J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization metric. Parameter count alone does not specify this geometry. The empirical scale comparisons also change pretrained models and learning-rate schedules; they neither measure the geometry nor identify the cause of cross-scale differences.

I.3 A REPLAY SIGNAL WHEN FRESH GROUPS SUPPLY NONE

Lemma I.4 (Fresh-group starvation and replay signal). *Fix a categorical policy with correctness probability P , and draw $G \geq 2$ fresh responses independently. For GRPO, Dr.GRPO, and the implemented centered MaxRL estimator, the probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

Every all-equal group has zero fresh task gradient. Thus h_G is the probability of an opportunity for a nonzero task gradient, rather than a lower bound on the gradient magnitude.

For a fixed bank with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ and $\rho_w > 0$. Extend w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p).$$

It vanishes only at $p = \bar{w}$. In particular, if $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$, even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes, as $p \rightarrow \mathbf{e}_b$ at one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$. The replay norm therefore stays positive as fresh task-signal opportunities vanish.

Proof. The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$. A mixed group requires both a correct and an incorrect sample; the union bound gives the two bounds GP and $G(1 - P)$. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$, proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k - 1)/k^2 = 1 - 1/k,$$

which gives the strictly positive limiting norm for $k \geq 2$. $\square$

The displayed replay vector averages the fixed bank. A stochastic exemplar draw has this vector only in expectation, and a nonzero logit coordinate can be annihilated by a degenerate neural Jacobian. Replay also need not increase the next mixed-group probability: $h_G(P)$ is largest at $P = 1/2$. The claim is that a banked target can supply a separate verified signal when the fresh task gradient is unavailable.

A bounded advantage based on fresh samples has a different boundary behavior. The next bound includes group-dependent detached scores, and distinguishes that estimator from exact entropy regularization.

Lemma I.5 (Bounded sampled scores have no probability-independent boundary force). *Condition on the training history, draw $A_1, \dots, A_G$ independently from p , and let a_i be any detached, possibly group-dependent advantages with $|a_i| \leq B$. At the on-policy point define*

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (e_{A_i} - p).$$

For every category b ,

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.

Proof. Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{A_i = b\} - p_b| = 2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

The bound does not prove inevitable collapse or rule out retention by a particular bounded estimator. An unsampled mode can still change through softmax normalization or shared parameters. Exact entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero.

I.4 FIXED-BANK REPLAY AND QUALITATIVE RETENTION

Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (11)$$

This is ordinary cross-entropy against the distribution that assigns mass $1/k$ to each banked mode and zero elsewhere. It is not conditioned on the bank: decreasing (11) can raise total bank mass as well as redistribute mass within the bank.

In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting; it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P) \nabla_z P - \rho \nabla_z R_{\mathcal{B}}(z). \quad (12)$$

Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas I.1 and I.2, c_G has a finite positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G - 1)/G$ and MaxRL has $c_G = [1 - (1 - P)^{G-1}]/P$. Define

$$F_{\mathcal{B}}(z) = \rho R_{\mathcal{B}}(z) - \Psi(P(z)). \quad (13)$$

Then (12) is exactly gradient flow $\dot{z} = -\nabla_z F_{\mathcal{B}}$.

Theorem I.6 (Verified replay prevents banked-mode extinction). *Suppose that after time T the nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective replay dose ρ . Under (12), define*

$$C_T = R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho}.$$

Then for every $b \in \mathcal{B}$ and every $t \geq T$,

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (14)$$

Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including asymptotically, even when the bank does not cover every correct mode. This is stronger than finite-time positivity, which already follows from finite softmax logits without replay.

Proof. Along the flow,

$$\frac{d}{dt} F_{\mathcal{B}}(z(t)) = -\|\nabla_z F_{\mathcal{B}}(z(t))\|_2^2 \leq 0.$$

Since $P(t) \leq 1$ and Ψ is increasing,

$$R_{\mathcal{B}}(z(t)) \leq R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho} = C_T.$$

Every $-\log p_b$ is nonnegative and their sum is $kR_{\mathcal{B}}$. Therefore $-\log p_b(t) \leq kC_T$ for every banked mode, which is exactly Equation (14) for every time after T . $\square$

The floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not the replay coefficient alone.

Full coverage and the categorical target.

Corollary I.7 (Conditional full-coverage categorical limit). *Assume the conditions of Theorem I.6 and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Let $u_c = 1/m$ on $\mathcal{C}$. Then the categorical mean flow converges to*

$$P(t) \rightarrow 1, \quad q(t) \rightarrow u.$$

Consequently

$$\min_{c \in \mathcal{C}} q_c(t) \rightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \rightarrow \log m,$$

and conditional correct-mode collapse is impossible.

Proof. For full coverage,

$$R_{\mathcal{C}} = \log m - \log P + \text{KL}(u||q). \quad (15)$$

The potential is bounded below by $-\Psi(1)$, so its descent gives $\int_T^\infty \|\nabla F_{\mathcal{C}}\|_2^2 dt < \infty$. The finite-group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded. Thus the Hessian of $F_{\mathcal{C}}$ and the velocity are bounded; the gradient along the trajectory is uniformly continuous. Square integrability then implies $\nabla F_{\mathcal{C}} \rightarrow 0$: recurring gradients of fixed positive size would otherwise contribute energy over intervals of a uniform minimum length.

Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness of this limit establish convergence of the probability vector. $\square$

Positive weights and the implemented length normalization. Uniformity of the target is separate from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w = -\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho_w}.$$

If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w\|q)$, and the independent-logit limit is $P \rightarrow 1$, $q \rightarrow w$. The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate equations forces $P = 1$, then each equation forces $p_c = w_c$.

A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$, so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-bank or stochastic neural training.

This distinction matters for the implemented mean-token exemplar loss. Even if each key were represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

Its effective categorical replay coefficient would be $\rho_w = \rho_{\text{LM}}A$. The full-coverage limit would therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual language model, $\pi_\theta(e_b \mid x)$ is only one contribution to the total key probability. Equal averaging of per-token exemplar scores does not imply uniform learned key probabilities; the categorical uniformity corollary is not that claim.

Full support is an additional premise, not a consequence of replay. In Python factors, prompts admit 16–3,600 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects only admitted banked modes; it supplies no probability floor for an unbanked key.

I.5 COMPLETE RESPONSES AND FINITE EVALUATION VISIBILITY

The learner scores a particular response string, while evaluation groups strings by execution key. Event containment provides a conditional bridge: one complete verified response is one way of producing its key. This bridge also makes the loss’s length factor explicit.

Lemma I.8 (Retention from a joint exemplar potential). *Fix finitely many verified complete-response exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j \mid x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$, every protected exemplar satisfies

$$p_\theta(b_j \mid x_j) \geq \pi_\theta(e_j \mid x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

Proof. The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative, so $-\log \pi_\theta(e_j \mid x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key b_j , which proves the stated lower bound on every protected key’s probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \Psi_x(1)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or score under another decoding law is insufficient. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K \delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero (Clopper & Pearson, 1934). For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

I.6 SCOPE RELATIVE TO ENTROPY, ADMISSION, AND NEURAL OPTIMIZATION

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q||u), \quad R_u(p) = \log m - \log P + \text{KL}(u||q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$ have the unique global optimum $P = 1, q = u$ on the simplex with $P > 0$, for positive coefficients. This follows directly from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets (Pereyra et al., 2017; Agarwal et al., 2021): replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions (Mei et al., 2020); our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes.

For noisy discrete updates, a valid extension would require control of the joint loss’s upward drift, optimizer bias, and accumulated error. A positive replay coefficient or small nominal learning rate does not establish these conditions. Current-gradient Adam moments, clipping, momentum, prompt scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not established the required joint-energy or update-error assumptions. The mathematical

contribution is the stated categorical mechanism and conditional exemplar argument; empirical support and longitudinal exemplar scores remain separate measurements.

J SOURCE INTEGRITY AND REPRODUCIBILITY

J.1 ADMISSIBLE SOURCES AND MISSING CHECKPOINTS

Falcon3-1B Countdown ReplayDr.GRPO seed 59 contains two complete four-draw terminal evaluation sequences with identical request identities but conflicting metrics. The integrity amendment excludes that source at every checkpoint and from every efficacy endpoint, paired contrast, and aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs: 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial intersections are descriptive and receive no confidence interval.

Trajectory records use registered sources and approved continuations or replacements, with superseded attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy and mode-count views use the same populations and gap rules. Supporting methods retain their explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the curves. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule, and source hashes.

J.2 DATA, MODELS, AND SCIENTIFIC RECORDS

The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row identities, and construction and admission records. Its loading guide and evaluation guide identify the reported splits and preserve auxiliary data separately. The model archive links each available model–domain–method–seed export to an immutable upload revision containing original weights, configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license. The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-weighting ablation, and supporting controls to their model families; the scientific-record index and restore guide identify preserved analysis inputs and SHA-256 verification procedures.

Each empirical figure has a machine-readable record identified in the figure manifest, containing its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper versions to their builders, records, and reproduction commands. Detailed source and recovery inventories remain in those records and the repository’s archived audit documents. Model availability may extend beyond the frozen analysis population; it does not change a figure’s source-admission or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories require the recorded evaluation checkpoints.

J.3 EXACT CONTINUATION AND VALIDATION

Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream state in Appendix D.4, in addition to model weights. State mismatch, non-finite values, missing completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures. Placement-only recovery may change hardware or wall time but must preserve the scientific update. Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints or silently incorporated from a newer archive.

K DISCLOSURES

AI USE STATEMENT

Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The authors reviewed and verified the resulting content and take responsibility for the final manuscript.

ETHICS STATEMENT

The benchmark and experiment-specific data are synthetic; this study collects no human-subject or personal data. More verified modes are not inherently better: a distinct execution may be risky or costly even when it passes the validator, and `distinct@K` does not measure downstream value. Systems that evaluate or optimize breadth should report correctness, invalidity, and application-specific costs alongside mode counts.

REPRODUCIBILITY STATEMENT

Claims are computed from versioned result files; missing final results and aggregate drift cause the build to fail. Appendix J records protocols, hashes, validations, and regeneration commands; Appendix B records split identities. The AI Use Statement above discloses generative-AI assistance.